Notes on Machine Learning

Course of Machine Learning

Master Degree in Computer Science

University of Rome "Tor Vergata"

Giorgio Gambosi

Academic Year: 2025-2026

Chapter 3

Loss functions & training

Loss function

The concept of a loss function lies at the very heart of machine learning. In its most general form, the loss function $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \mapsto \mathbb{R}$ serves as a quantitative measure of the **cost** incurred when we refer to a predicted value y instead of the true target value t for any subsequent decision or action. Here, both y and t are elements belonging to the target space $\mathcal{Y}$, which could represent real numbers in regression problems, discrete class labels in classification tasks, or more complex structured outputs in other learning scenarios.

To understand why we need such a measure, consider that in supervised learning our goal is not merely to generate predictions, but to generate *good* predictions—ones that closely align with the true underlying relationship between inputs and outputs. The loss function provides precisely this measure of quality. Specifically, it quantifies how well the prediction function h performs on a given input-output pair. For an input $\mathbf{x}$ with corresponding true output y, we evaluate the quality of our prediction through:

$$\mathcal{R}(\mathbf{x}, y) = L(h(\mathbf{x}), y) \tag{3.1}$$

This individual loss tells us how far off we are for a single prediction, but in practice we work with entire datasets. Therefore, the loss function becomes a fundamental component of what we call the **empirical risk**, which aggregates the loss across all examples in our training set. The empirical risk is defined as the average value of the loss function applied to all predicted value-target value pairs in the training set $\mathcal{T}$:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(x,t)\in\mathcal{T}} L(h(x), t) \tag{3.2}$$

This quantity serves as our primary indicator of model quality, at least with respect to the available training data. It answers the question: "On average, how much error does our prediction function make?" The lower this value, the better our model fits the training data. However, we must always keep in mind that minimizing empirical risk on training data is not our ultimate goal—we truly care about performance on unseen data, which relates to the concept of generalization.

During the training phase, our objective is to minimize the empirical risk with respect to the prediction function h. In practice, this means we work with parametric functions $h = h_\theta$, where the behavior of h is completely determined by a set of parameters $\boldsymbol{\theta} = (\theta_0, \theta_1, \dots, \theta_d)$. The training process can thus be viewed as a search through parameter space for the configuration that yields the smallest empirical risk.

This minimization problem can be equivalently formulated as minimizing the overall loss:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \sum_{i=1}^{n} L_i(\boldsymbol{\theta}) \tag{3.3}$$

which represents the sum of individual loss functions $L_i = L(\boldsymbol{\theta}; \mathbf{x}_i, t_i)$ computed for each data point $(\mathbf{x}_i, t_i)$ in our training set. The notation emphasizes that the loss depends on both the parameters $\boldsymbol{\theta}$ and the specific training example being considered.

Loss function minimization approaches

Having defined what we want to minimize, we now face the crucial question: how can we actually tackle the problem of minimizing the loss function in practice? This is not merely a theoretical concern—the efficiency and effectiveness of our optimization strategy directly determines whether we can successfully train our models.

Ideally, our goal is to find a **global minimum** of the loss function, which would represent the best possible set of parameters for our model across the entire parameter space. A global minimum is a point where the loss is lower than at any other point in the parameter space, meaning we have found the truly optimal configuration.

One classical approach to finding minima relies on fundamental results from calculus. The key insight is that at any minimum (whether local or global), the gradient—that is, the vector of all partial derivatives—must equal zero. This is because if the gradient were non-zero at some point, we could move in the direction opposite to the gradient and further decrease the function value, contradicting the assumption that we are at a minimum.

Mathematically, we seek to solve:

$$\nabla \mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \mathbf{0} \tag{3.4}$$

Let us clarify the notation here. The symbol ∇ denotes the gradient operator, which, given a multivariate function $f(x_1, \dots, x_m)$, returns the vector of its partial derivatives with respect to each variable:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_m} \end{pmatrix} \tag{3.5}$$

An important geometric interpretation helps build intuition: the gradient at any given point $\overline{x}_1, \dots, \overline{x}_m$ in the domain space of f is a vector pointing in the direction of steepest ascent from that point. Conversely, the negative of the gradient points in the direction of steepest descent—a fact we will exploit later.

Setting the gradient to $\mathbf{0}$ means solving a system of equations where each component is set to zero:

$$\frac{\partial}{\partial \theta_i} \mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = 0 \quad \forall i \tag{3.6}$$

Each equation in this system represents the condition that the loss function does not increase (to first order) when we make an infinitesimal change to parameter θ_i while holding all other parameters fixed. Finding values of $\boldsymbol{\theta}$ that satisfy all these equations simultaneously would give us candidate points where the loss could be minimal.

However, this elegant mathematical approach faces several significant practical challenges that limit its applicability, especially in modern machine learning contexts. First and foremost, this system of equations typically has multiple solutions. These solutions include not only local minima (which are our desired outcomes) but also local maxima (which are the worst possible points to stop at) and saddle points (which are neither minima nor maxima but rather inflection points that are critical nonetheless). Without additional analysis, we cannot distinguish between these different types of critical points.

Moreover, and perhaps more critically, in many practical cases—especially those involving complex models like neural networks—solving these equations analytically is either extremely difficult or outright impossible. The loss functions we encounter in deep learning are highly non-linear, involving compositions of many non-linear activation functions, and analytical solutions simply do not exist in closed form. This necessitates the use of numerical optimization methods.

Gradient descent

Given the challenges of analytical approaches, we turn to iterative numerical methods that can approximate solutions. Among these, **gradient descent** stands out as one of the most fundamental and widely used algorithms in machine learning. While it may not guarantee finding the global minimum, it provides a practical and computationally efficient way to find a local minimum of the empirical risk function $\overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta})$.

The gradient descent algorithm follows a beautifully simple principle: if we want to minimize a function, we should move in the direction where the function decreases most rapidly. Since the gradient points in the direction of steepest ascent, moving in the opposite direction—that is, in the direction of the negative gradient—takes us toward steepest descent.

The process begins by initializing our parameters at some starting point, which we denote as $\boldsymbol{\theta}^{(0)} = (\theta_0^{(0)}, \theta_1^{(0)}, \dots, \theta_d^{(0)})$. This initialization is often random or based on some heuristic, and it comes with an initial error value given by:

$$\overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta}^{(0)}). \tag{3.7}$$

This initial error tells us how poorly our randomly initialized model performs—typically quite poorly indeed! The goal of gradient descent is to iteratively improve upon this initial configuration.

The iterative procedure works by repeatedly modifying the current parameter values $\boldsymbol{\theta}^{(i-1)}$ in the direction of **steepest descent** of the empirical risk function $\overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta})$. Specifically, this means moving in the opposite direction of the gradient of the empirical risk evaluated at our current position $\boldsymbol{\theta}^{(i-1)}$. We take a step down the slope of the error surface, moving toward (we hope) a valley where the error is minimized.

At each iteration i of the algorithm, each parameter $\theta_j^{(i-1)}$ is updated according to the following rule:

$$\theta_j^{(i)} := \theta_j^{(i-1)} - \eta \frac{\partial}{\partial \theta_j} \overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}^{(i-1)}} = \theta_j^{(i-1)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \frac{\partial}{\partial \theta_j} L(h_{\boldsymbol{\theta}}(\mathbf{x}), t)\Big|_{\boldsymbol{\theta}^{(i-1)}}, \tag{3.8}$$

Let us unpack this equation carefully, as it encodes the entire essence of gradient descent. The parameter η is called the **learning rate**, and it controls the step size during each update. Think of it as determining how aggressive we are in following the gradient's direction: a large learning rate means we take big steps, potentially converging faster but risking overshooting the minimum; a small learning rate means we take tiny steps, converging more reliably but possibly very slowly. The partial derivative term tells us the direction and rate of change of the loss with respect to this particular parameter, and the negative sign ensures we move downhill rather than uphill.

In matrix form, which provides a more compact notation, this update rule can be written as:

$$\boldsymbol{\theta}^{(i)} := \boldsymbol{\theta}^{(i-1)} - \eta\, \nabla \overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta})\Big|_{\boldsymbol{\theta}^{(i-1)}} = \boldsymbol{\theta}^{(i-1)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \nabla L(h_{\boldsymbol{\theta}}(\mathbf{x}), t)\Big|_{\boldsymbol{\theta}^{(i-1)}}. \tag{3.9}$$

This formulation makes clear that we update all parameters simultaneously based on the full gradient vector computed from all training examples.

While this method allows us to approximate a local minimum in a computationally tractable manner, the outcome depends heavily on several factors, particularly the initial parameter values chosen. The loss landscape in high-dimensional parameter spaces can be extraordinarily complex, with many valleys, peaks, and plateaus. Different starting points can lead us to entirely different local minima, some of which may be much better than others.

This observation leads us to several key challenges and questions that practitioners must grapple with:

- We are fundamentally interested in finding a global minimum—the absolute best configuration of parameters—but gradient descent can easily get stuck at local minima. How can we encourage exploration of the parameter space to avoid settling for poor local optima?
- How do we handle saddle points, which are critical points (where the gradient is zero) that are neither minima nor maxima? In high-dimensional spaces, saddle points are actually far more common than local minima, and gradient descent can slow down dramatically near these points, even though they are not minima at all.
- How quickly does the method converge, and what factors influence its speed? The convergence rate depends on the learning rate, the conditioning of the loss function (how stretched or round the error surface is), and the quality of our initialization.

- What happens if the learning rate is too large or too small? Can we adapt it during training to achieve better performance?

These challenges make the minimization process considerably more complex than it might initially appear, but gradient descent and its many variants remain among the most widely used methods in practice for parameter optimization. Modern deep learning has developed numerous sophisticated extensions—including momentum methods, adaptive learning rates, and second-order approximations—that address many of these challenges while maintaining computational efficiency.

Convexity

Having explored the challenges of optimization, we now turn to a mathematical property that, when present, dramatically simplifies our task: convexity. Understanding convexity is crucial because it tells us when optimization becomes significantly easier—when we can be confident that any local minimum we find is actually the global minimum.

Let us begin with the geometric definition. A set of points $S \subset \mathbb{R}^d$ is said to be **convex** if and only if for any two points $\mathbf{x}_1, \mathbf{x}_2 \in S$ and any scalar $\lambda \in (0, 1)$, we have:

$$\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in S \tag{3.10}$$

In intuitive terms, this means that if we take any two points in the set and draw a straight line segment connecting them, every point on that line segment must also belong to the set. Convex sets have no "holes" or "indentations"—they are, in a sense, solid and without concavities.

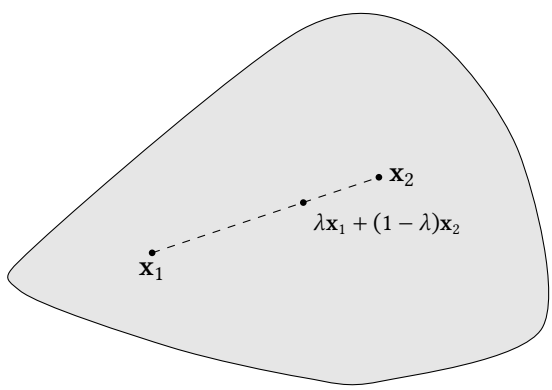


Figure 3.1. Convex set

Now, extending this concept from sets to functions, we say that a function $f(\mathbf{x})$ is **convex** if and only if the set of points lying on or above the graph of the function forms a convex set. More precisely, for all points $\mathbf{x}_1, \mathbf{x}_2$

in the domain and all $\lambda \in (0, 1)$, the following inequality must hold:

$$f(\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1-\lambda) f(\mathbf{x}_2) \tag{3.11}$$

This elegant condition has a beautiful geometric interpretation: if we take any two points on the graph of f and connect them with a straight line segment, that line segment must lie on or above the graph of the function. In other words, the function never curves upward more than a straight line would—it has no concave regions where it dips down.

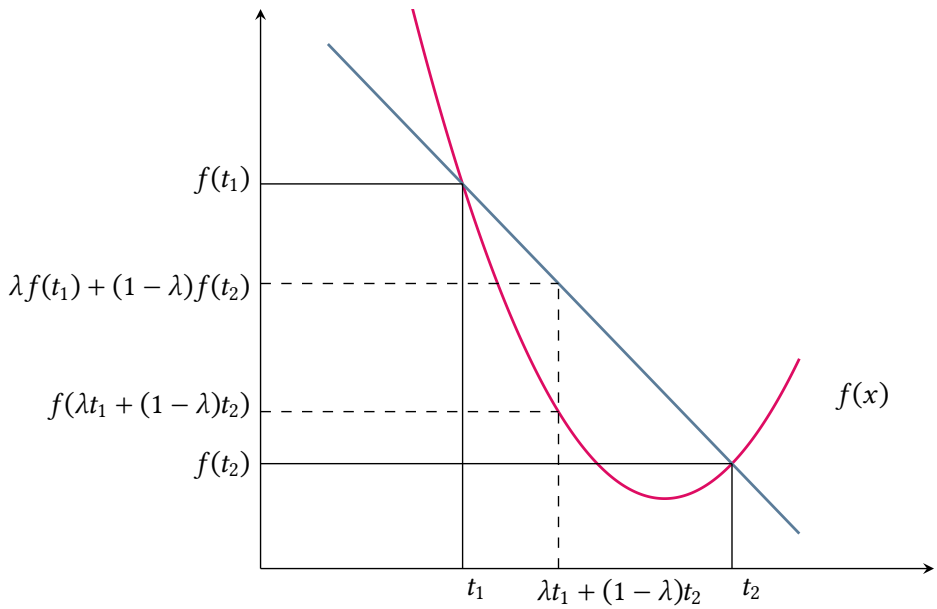


Figure 3.2. Convex function

An even stronger property is **strict convexity**, where the inequality becomes strict. A function $f(\mathbf{x})$ is strictly convex if and only if for all distinct points $\mathbf{x}_1 \neq \mathbf{x}_2$ and all $\lambda \in (0, 1)$, we have:

$$f(\lambda \mathbf{x}_1 + (1-\lambda)\mathbf{x}_2) < \lambda f(\mathbf{x}_1) + (1-\lambda) f(\mathbf{x}_2) \tag{3.12}$$

The difference between convexity and strict convexity is subtle but important: strictly convex functions curve strictly upward—they cannot have any flat regions or linear segments.

Why does convexity matter so profoundly for optimization? The answer lies in a remarkable theoretical result: *if $f(\mathbf{x})$ is a convex function, then any local minimum of f is also a global minimum*. This means that if we use gradient descent and it converges to a local minimum (where the gradient is zero), we can be confident that we have found the best possible solution. There are no hidden valleys elsewhere in the parameter space with lower function values.

The situation is even better for strictly convex functions: if f is **strictly convex**, there exists *only one* local minimum for f, and it is necessarily the global minimum. This uniqueness property is extraordinarily powerful—it means that regardless of where we initialize gradient descent, we will always converge to the same optimal solution (assuming appropriate learning rates and sufficient iterations).

Therefore, when our loss function $\mathcal{L}(\boldsymbol{\theta}; \mathcal{T})$ is convex, solving the system of equations:

$$\nabla \mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \mathbf{0} \tag{3.13}$$

is guaranteed to provide us with a global minimum, making the optimization problem tractable both theoretically and practically.

A particularly important special case occurs when $f(\mathbf{x})$ is a quadratic function—that is, when it can be written as a second-degree polynomial in the variables. Quadratic functions are always convex (and strictly convex if their matrix of second derivatives is positive definite), and they admit closed-form solutions. This favorable situation arises in several classical machine learning models, including linear regression with squared error loss and certain formulations of logistic regression.

Unfortunately, this convenient property does not hold for more complex models, particularly neural networks with non-linear activation functions. The loss surface of a deep neural network is highly non-convex, with potentially many local minima, saddle points, and plateaus. This non-convexity is one reason why training deep networks remains challenging and why much research focuses on developing better optimization algorithms and initialization strategies.

To leverage convexity when possible, we can exploit some useful mathematical properties. Let us recall two fundamental results about convex functions:

1. The sum of (strictly) convex functions is itself (strictly) convex. This is straightforward to verify: if f and g are both convex, then their sum $h = f + g$ satisfies the convexity inequality because both f and g individually satisfy it.
2. The product of a (strictly) convex function and a positive constant is (strictly) convex. Scaling preserves the geometric property of curving upward.

These properties have immediate implications for our empirical risk. Recall that:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t) \propto \sum_{(\mathbf{x},t)\in\mathcal{T}} L(\boldsymbol{\theta}; \mathbf{x}, t) \tag{3.14}$$

The empirical risk is simply a scaled sum of individual loss terms evaluated at each training example. Applying the properties above, we can draw several important conclusions:

- If each individual loss function $L(\boldsymbol{\theta}; \mathbf{x}, t)$ is (strictly) convex with respect to the parameters $\boldsymbol{\theta}$, then the overall empirical risk is also (strictly) convex. This follows immediately from the fact that we are forming a weighted sum (with positive weights) of convex functions.

- If $L(\boldsymbol{\theta}; \mathbf{x}, t)$ is convex, then any local minimum of the empirical risk is guaranteed to be a global minimum. This means gradient descent, if it converges, will find the optimal solution regardless of initialization.
- If $L(\boldsymbol{\theta}; \mathbf{x}, t)$ is strictly convex, then there exists a unique minimum of the empirical risk. Not only is any local minimum global, but there is exactly one such minimum. This provides maximal certainty in our optimization procedure.

These results highlight why much effort in machine learning theory and practice goes into designing loss functions and model architectures that preserve convexity, or at least approximate it sufficiently well. When convexity can be achieved or approximated, optimization becomes vastly more reliable and predictable, allowing us to focus on other aspects of model design and deployment.

Some common loss functions

Having established the theoretical framework for loss functions and their optimization, we now turn our attention to the practical question: what specific loss functions should we use in different machine learning scenarios? The choice of loss function is not merely a technical detail—it profoundly influences both the optimization landscape we must navigate and the type of solution our model will ultimately learn. Different loss functions encode different assumptions about what constitutes a "good" prediction and how severely we should penalize different types of errors.

In this section, we examine the most commonly used loss functions in supervised learning, organized by the type of learning task they are designed for. We begin with regression problems, where we predict continuous values, and then move to classification problems, where we assign discrete labels.

Loss functions for regression

Let us first consider the case of **regression**, where our goal is to predict a continuous numerical output. The regression setting has several defining characteristics that shape our choice of loss function:

- Both the true target value t and our prediction $h(\mathbf{x})$ are real-valued numbers, typically in $\mathbb{R}$ or some bounded subset thereof. This means we are working in a continuous output space rather than with discrete categories.
- The natural notion of error in this setting relates to some form of distance measure between points in this continuous space. We want to quantify "how far" our prediction is from the truth, and different distance measures lead to different loss functions with distinct properties.

The choice of distance measure is not arbitrary—it reflects our assumptions about the nature of errors and their consequences. Should large errors be penalized much more severely than small ones? Should we treat positive and negative errors symmetrically? These considerations guide our selection of an appropriate loss function.

Quadratic loss

The most ubiquitous and widely used loss function for regression tasks is the **quadratic loss**, also known as squared error or L_2 loss. It is defined simply as:

$$L(y, t) = (y - t)^2 \tag{3.15}$$

This deceptively simple formula has profound implications. The quadratic loss measures the squared Euclidean distance between prediction and target, meaning that errors are penalized quadratically—an error twice as large incurs four times the loss. This has the effect of making the optimization procedure particularly sensitive to outliers, as we shall discuss.

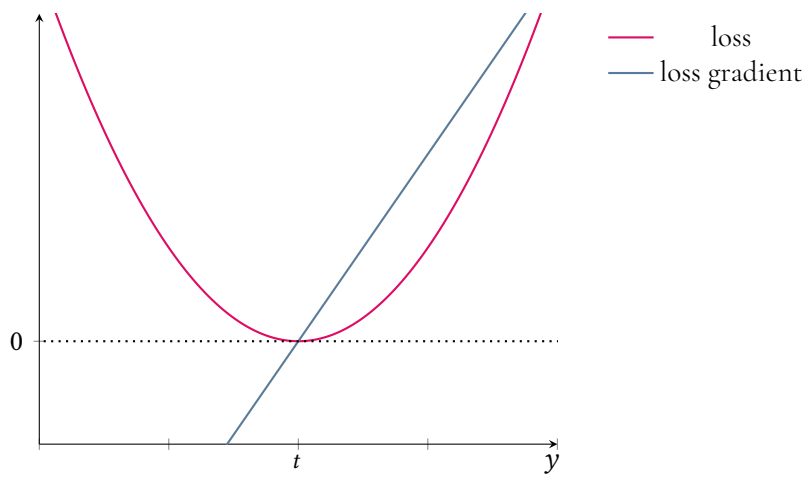


Figure 3.3. Quadratic loss function and its gradient. Note the parabolic shape and the linear gradient.

When we apply quadratic loss across our entire training set, we obtain the empirical risk in a particularly elegant form:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} (h(\mathbf{x}) - t)^2 \tag{3.16}$$

This quantity, often called the **mean squared error** (MSE), represents the average squared deviation between our predictions and the true values across all training examples. Minimizing this quantity is equivalent to finding the prediction function that best fits the training data in the least-squares sense.

In the particularly common case of **linear regression**, where we assume the underlying relationship between inputs and outputs is linear, the prediction is performed by means of a linear function $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$, where $\mathbf{w}$ represents a weight vector and b is a bias term. Substituting this into our empirical risk formula results in an overall loss function to be minimized:

$$\mathcal{L}(\mathbf{w}, b; \mathcal{T}) = \sum_{(\mathbf{x},t)\in\mathcal{T}} (\mathbf{w}^T\mathbf{x} + b - t)^2 \tag{3.17}$$

This formulation has several extraordinarily favorable mathematical properties that make it the workhorse of classical statistical learning:

- Since the quadratic function is strictly convex, the overall loss $\mathcal{L}(\mathbf{w}, b; \mathcal{T})$ has only one local minimum, which is necessarily the global minimum. This guarantees that any optimization procedure that successfully finds a local minimum has found the best possible solution.
- The gradient of the loss with respect to the parameters is linear, which makes optimization particularly tractable. Specifically, we can compute:

$$\frac{\partial}{\partial w_i}\mathcal{L}(\mathbf{w}, b; \mathcal{T}) = \sum_{(\mathbf{x},t)\in\mathcal{T}} (\mathbf{w}^T\mathbf{x} + b - t)x_i \qquad \frac{\partial}{\partial b}\mathcal{L}(\mathbf{w}, b; \mathcal{T}) = \sum_{(\mathbf{x},t)\in\mathcal{T}} (\mathbf{w}^T\mathbf{x} + b - t) \tag{3.18}$$

  where x_i denotes the i-th component of the input vector $\mathbf{x}$. The linearity of these gradients means that gradient descent will converge smoothly without oscillations or instabilities.
- In fact, for linear regression with quadratic loss, we can even derive a closed-form analytical solution by setting these gradients to zero and solving the resulting system of linear equations. This is the famous **normal equations** approach, which provides an exact solution without requiring iterative optimization.

Despite these attractive properties, quadratic loss is not without its drawbacks. Its primary weakness is that it is not robust to outliers. Because errors are squared, a single data point with a very large error can dominate the overall loss, causing the fitted model to be pulled dramatically toward that outlier at the expense of fitting the bulk of the data well. This sensitivity can be problematic in real-world datasets where outliers or measurement errors are common.

Absolute loss

While quadratic loss is mathematically convenient and has elegant properties, its lack of robustness to outliers motivates the search for alternatives. One natural alternative is the **absolute loss**, also known as L_1 loss or mean absolute error:

$$L(y, t) = |y - t| \tag{3.19}$$

The absolute loss measures the absolute value of the difference between prediction and target, penalizing errors linearly rather than quadratically. This seemingly small change has significant practical consequences: whereas quadratic loss penalizes an error of magnitude ϵ with cost ϵ^2, absolute loss penalizes it with cost ϵ. This means that large errors grow linearly rather than quadratically, making absolute loss much more robust to outliers.

An important feature of absolute loss is immediately apparent from examining its gradient: the gradient is piecewise constant, taking the value -1 when $y < t$ and $+1$ when $y > t$. This constant gradient magnitude means that gradient descent will take steps of uniform size regardless of how far the current prediction is from the target

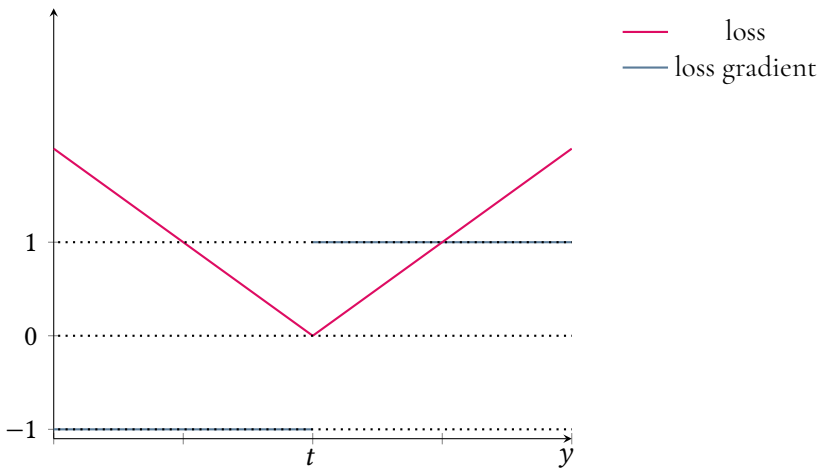


Figure 3.4. Absolute loss function and its gradient. The V-shape is characteristic, with constant gradient magnitude.

(modulo the learning rate). This is in stark contrast to quadratic loss, where the gradient grows linearly with the error, causing larger steps when predictions are far from targets.

The absolute loss is convex, which ensures that any local minimum is global, but it is not strictly convex (notice the flat gradient regions). This means there may be multiple optimal solutions, though in practice this rarely causes difficulties. A more significant practical challenge is that the gradient is undefined at $y = t$, where the function has a sharp corner. However, this can be handled using subgradient methods, which we will discuss in more detail when examining classification losses.

Huber loss

Given the trade-offs between quadratic and absolute loss—the former being smooth and easy to optimize but sensitive to outliers, the latter being robust but having discontinuous derivatives—a natural question arises: can we design a loss function that combines the best of both worlds? The **Huber loss** provides exactly such a compromise.

The Huber loss is quadratic for small errors (like quadratic loss) but transitions to linear growth for large errors (like absolute loss). This hybrid behavior is achieved through the following piecewise definition:

$$L(t, y) = \begin{cases} \frac{1}{2}(t - y)^2 & |t - y| \leq \delta \\ \delta(|t - y| - \frac{\delta}{2}) & |t - y| > \delta \end{cases} \tag{3.20}$$

The parameter δ controls the transition point between quadratic and linear behavior. For errors smaller than δ in magnitude, the loss behaves quadratically, providing smooth gradients that facilitate efficient optimization. For errors larger than δ, the loss grows linearly, providing robustness against outliers by preventing them from dominating the overall loss.

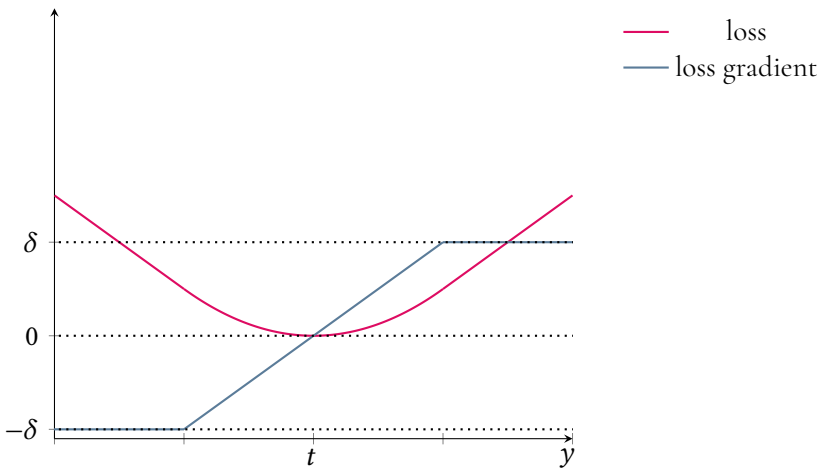


Figure 3.5. Huber loss combines quadratic behavior near zero with linear growth for large errors.

The Huber loss represents an elegant compromise that is particularly valuable in practical applications where we expect our data to contain some outliers but still want to benefit from the smooth optimization properties of quadratic loss for the majority of well-behaved data points. The choice of δ becomes a hyperparameter that can be tuned based on the expected scale and frequency of outliers in the specific application domain.

Loss functions for classification

We now shift our attention from regression to classification, where the prediction task is fundamentally different in nature. In classification, we assign inputs to discrete categories or classes rather than predicting continuous values. This discrete structure necessitates a different approach to measuring error and, consequently, different loss functions.

There are essentially two distinct approaches to classification, which lead to different types of loss functions:

- **Direct class prediction**: The prediction function returns a specific class label directly. In this case, we measure error based on the coincidence (or lack thereof) between the predicted class and the true class.
- **Probabilistic prediction**: The prediction function returns a probability distribution over the set of possible classes. Here, we measure error based on the cumulative difference between the predicted probability distribution and the true distribution (which assigns probability 1 to the correct class and 0 to all others).

These two approaches imply fundamentally different definitions of what constitutes an error:

- In the first case, we care only about whether the predicted class matches the real class—there is no notion of being "almost correct."
- In the second case, we care about the confidence of predictions—a prediction that assigns high probability to the correct class is better than one that barely favors it, even if both make the same final classification decision.

For concreteness and clarity, we will focus on the **binary classification** case, where there are exactly two classes. We identify these classes with target values $t \in \{-1, +1\}$, a convention that proves mathematically convenient. The extension to multi-class classification follows similar principles but with added notational complexity.

In what follows, we will assume that our classifier produces a real-valued output $y = h(\mathbf{x})$, which can be interpreted as a confidence score. The sign of y determines the predicted class ($+1$ if $y > 0$, -1 if $y < 0$), while the magnitude $|y|$ represents the confidence of that prediction. This framework, where a single real number encodes both the predicted class and the prediction confidence, is extremely common and underlies many important classification algorithms.

0/1 loss

The most intuitive and "natural" loss function for classification problems is the **0/1 loss**, which directly encodes the notion of correctness: we incur a loss of 1 for an incorrect prediction and a loss of 0 for a correct prediction. Formally:

$$L(t, y) = \begin{cases} 1 & \text{sgn}(y) \neq t \\ 0 & \text{sgn}(y) = t \end{cases} \tag{3.21}$$

where $\text{sgn}(y)$ denotes the sign of y, defined as $+1$ if $y > 0$ and -1 if $y \leq 0$.

Using our convention where predictions are real numbers y and targets are in $\{-1, +1\}$, we can express this more compactly. Observe that $ty < 0$ precisely when the signs of t and y differ—that is, when we have made an incorrect prediction. Thus, we can write:

$$L(t, y) = \mathbb{1}[ty < 0] \tag{3.22}$$

where $\mathbb{1}[\cdot]$ is the indicator function, equal to 1 when its argument is true and 0 otherwise.

The 0/1 loss has obvious intuitive appeal—it directly measures classification accuracy, which is often what we ultimately care about. When we minimize the empirical risk under 0/1 loss, we are explicitly minimizing the classification error rate on our training data. However, despite this conceptual clarity, using 0/1 loss in practice is highly problematic for several fundamental reasons:

- **Non-convexity**: The 0/1 loss is not convex. This means that our loss landscape may have many local minima, and gradient-based optimization methods offer no guarantee of finding the global minimum. Worse, different initializations may lead to vastly different solutions.
- **Non-smoothness**: The function is not smooth—it has a discontinuous jump at $ty = 0$. The first derivative is undefined at this point, making it impossible to apply standard calculus-based optimization techniques.
- **Zero gradients almost everywhere**: Perhaps most critically, the gradient of 0/1 loss is zero almost everywhere (specifically, everywhere except at $ty = 0$, where it is undefined). This means gradient descent cannot make any progress—the gradient provides no information about which direction to move in parameter space to reduce the loss.

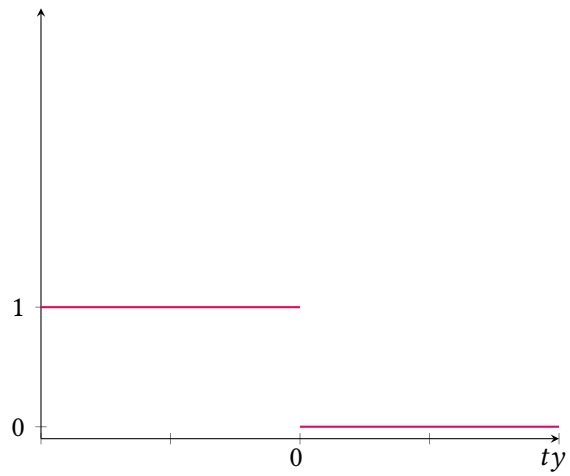


Figure 3.6. The 0/1 loss is a step function that assigns uniform cost to all errors.

- **Computational intractability**: If we assume a linear prediction function $y = \mathbf{w}^T\mathbf{x} + b$, the empirical risk becomes:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \mathbb{1}[(\mathbf{w}^T\mathbf{x} + b)t < 0] \tag{3.23}$$

  The problem of finding the values $\mathbf{w}, b$ that minimize the overall number of classification errors is NP-hard. This means that, unless P=NP (which is widely believed to be false), there exists no polynomial-time algorithm guaranteed to find the optimal solution. The problem remains computationally intractable even for moderately sized datasets.

These severe limitations make direct optimization of 0/1 loss impractical for most real-world applications. This fundamental difficulty motivates the development of alternative loss functions that approximate 0/1 loss but possess better mathematical properties.

Convex surrogate loss functions

Given the intractability of directly optimizing 0/1 loss, a central question in classification theory is: what alternative loss functions can we use that are both mathematically tractable and still produce good classifiers? The answer lies in **convex surrogate loss functions**, which have become the foundation of modern classification algorithms.

A surrogate loss function is one that serves as a substitute for 0/1 loss during optimization. For a surrogate to be useful, it should satisfy several key properties:

- **Upper bound property**: The surrogate should approximate 0/1 loss **from above**, meaning that the surrogate loss is always at least as large as the 0/1 loss. This ensures that minimizing the surrogate will also tend to

minimize the actual classification error. Formally, for all t and y, we require:

$$L_{\text{surrogate}}(t, y) \geq \mathbb{1}[ty < 0] \tag{3.24}$$

- **Convexity**: The surrogate should be convex with respect to the prediction y (and consequently with respect to the model parameters). This guarantees that any local minimum we find is also a global minimum, making optimization reliable and predictable.
- **Smoothness**: The surrogate should be smooth, with well-defined derivatives almost everywhere. This allows us to use gradient-based optimization methods effectively, with gradients that provide meaningful information about which direction to move in parameter space.

Different surrogate loss functions make different trade-offs in how they approximate 0/1 loss. A key distinction among them lies in the relevance or weight they assign to different types of errors. Some surrogates penalize all errors equally, while others heavily penalize certain types of errors (such as very confident but incorrect predictions) while being more lenient toward others (such as correct predictions with low confidence).

In what follows, we examine the most important and widely used surrogate loss functions, analyzing their properties and understanding when each is most appropriate.

Perceptron loss

The **perceptron loss** provides one of the simplest modifications to 0/1 loss. Recall that 0/1 loss assigns the same cost of 1 to every misclassified example, regardless of how wrong the prediction is. The perceptron loss refines this by taking into account the magnitude of the error.

The key insight is the following: if we assume our prediction y is a real value, then for a misclassified element (where $ty < 0$), we can measure the severity of the error using the value $-ty$. This quantity is positive for misclassified examples and increases as the prediction becomes more confidently wrong. The perceptron loss is defined as:

$$L(t, y) = \max(0, -ty) \tag{3.25}$$

Let us unpack what this means:

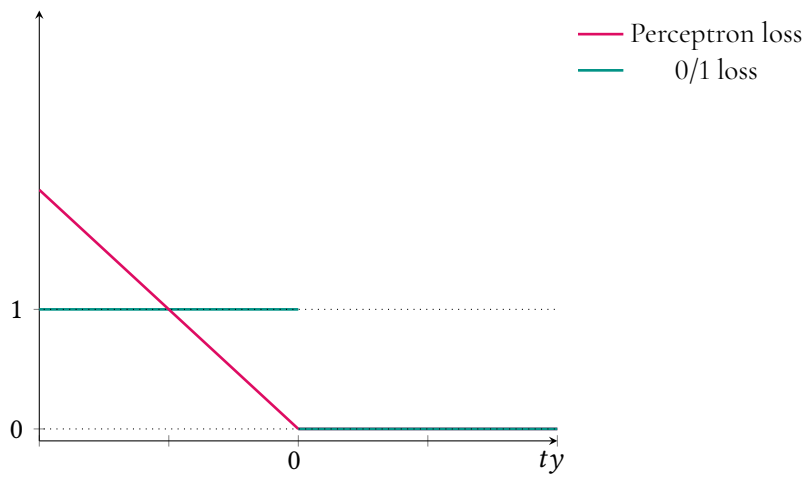
- For correctly classified elements where $ty > 0$, the loss is zero. The perceptron loss does not penalize correct predictions at all, regardless of how confident they are.
- For incorrectly classified elements where $ty < 0$, the loss equals $-ty = |ty|$, which is the absolute magnitude of the product ty. The more negative this product (meaning the more confident the wrong prediction), the higher the penalty.

The main philosophical difference from 0/1 loss is this: the perceptron loss distinguishes between different types of errors based on their severity. An example that is predicted to be strongly in class $+1$ (large positive

y) when it actually belongs to class -1 (so $t = -1$) incurs a much larger loss than an example whose prediction is only slightly positive. This makes intuitive sense—we should care more about fixing egregious errors than marginal ones.

From a mathematical perspective, the perceptron loss has several important properties:

- It is continuous everywhere
- Its gradient is continuous almost everywhere (specifically, everywhere except at $ty = 0$)
- It is convex, ensuring that local minima are global
- However, it is not strictly convex (it has linear regions)
- Importantly, it is **not** a surrogate loss in the strict sense—it does not always upper bound the 0/1 loss. For correctly classified examples near the decision boundary, the perceptron loss is 0 while 0/1 loss is also 0, but the perceptron loss doesn't provide the same guarantee for all correctly classified points



The perceptron loss is historically significant as it underlies the classical perceptron algorithm, one of the earliest learning algorithms developed in the 1950s. While simple, it demonstrates the key principle of using a tractable surrogate to approximate the intractable 0/1 loss.

Hinge loss

The **hinge loss** represents a refinement of the perceptron loss and is the foundation of one of the most important classification algorithms: the Support Vector Machine (SVM). Like the perceptron loss, it is linear for misclassified examples, but it introduces a crucial modification: it also penalizes correctly classified examples that are too close to the decision boundary.

The hinge loss is defined as:

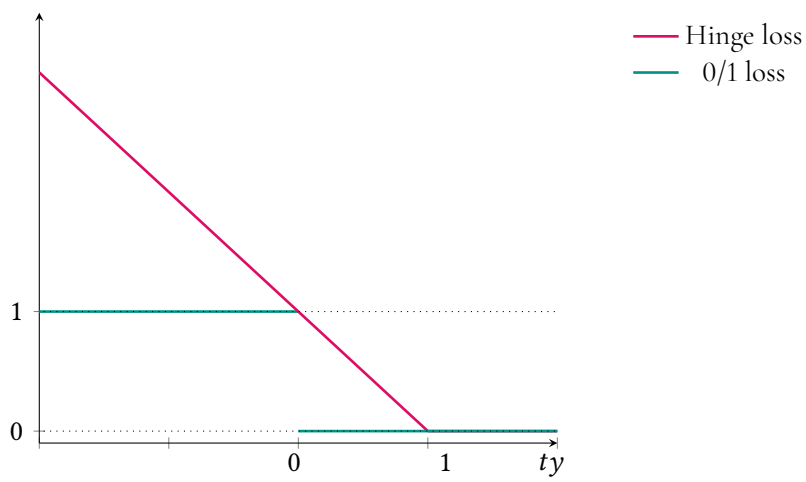
$$L(t, y) = \max(0, 1 - ty) \tag{3.26}$$

Compare this carefully to the perceptron loss $\max(0, -ty)$. The critical difference is the constant 1 instead of 0 inside the expression. This seemingly minor change has profound implications:

- The loss is zero only when $ty \geq 1$, not merely when $ty > 0$. This means that correct predictions (where $ty > 0$) can still incur a penalty if they are not sufficiently confident.
- For misclassified examples where $ty < 0$, the loss is $1 - ty$, which grows linearly as the prediction becomes more wrong.
- For correctly classified examples where $0 < ty < 1$, the loss is $1 - ty > 0$. These are correct but "weak" predictions—the classifier has made the right choice but without much confidence. The hinge loss penalizes such weak correct predictions, encouraging the learner to push them further from the decision boundary.
- Only when $ty \geq 1$ does the loss become zero. This means the classifier is encouraged to make confident correct predictions, with a "margin" of safety between the prediction and the decision threshold.

From a mathematical standpoint, hinge loss possesses several valuable properties:

- It is continuous everywhere
- Its gradient is continuous almost everywhere (except at $ty = 1$, where it has a corner)
- It is convex (though not strictly convex, due to the flat region where $ty > 1$)
- Crucially, it is a proper surrogate loss—it provides an upper bound on 0/1 loss for all values of ty



The hinge loss is used in Support Vector Machine training, where the goal is not just to classify examples correctly but to find a decision boundary that maximizes the margin—the distance between the boundary and the nearest examples from each class. The penalty for weak correct predictions (where $0 < ty < 1$) is precisely what encourages this margin-maximizing behavior.

A technical note on differentiation: The hinge loss $L_H(t, y) = \max(0, 1 - ty)$ is not differentiable with respect to y at the point $ty = 1$, where the function has a corner. Similarly, the perceptron loss has this issue at $ty = 0$. At these points, the derivative is undefined in the classical sense.

For example, computing the derivative of hinge loss yields:

$$\frac{\partial}{\partial y} L_H = \begin{cases} -t & ty < 1 \\ 0 & ty > 1 \\ \text{undefined} & ty = 1 \end{cases} \tag{3.27}$$

This non-differentiability poses a challenge if we want to apply gradient descent. However, we can work around this issue using the concept of a **subgradient**.

Subgradients

To understand subgradients, let us first recall what the gradient does for differentiable convex functions. Given a convex function f and a point x where f is differentiable, the gradient $\nabla f|_x$ defines a linear function (specifically, the tangent hyperplane at x) that provides a lower bound for f everywhere:

$$f(x') \geq f(x) + \nabla f|_x \cdot (x' - x) \quad \text{for all } x' \tag{3.28}$$

This is a fundamental property of convex functions—the tangent line at any point lies below the function. Geometrically, this means that if we stand at point x on the graph of f, the function never dips below the plane defined by the gradient at that point.

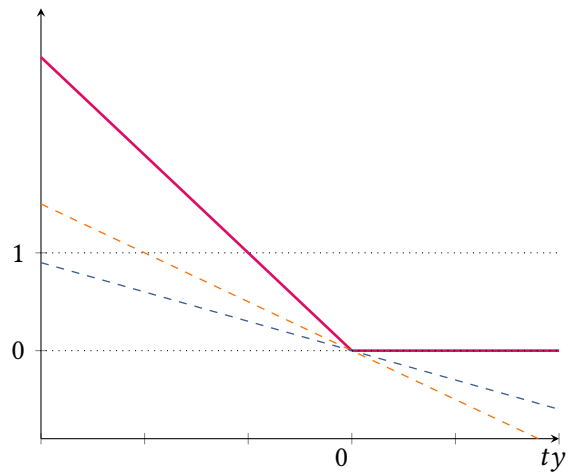
Now, what happens at a singular point x where f is not differentiable and the classical gradient $\nabla f|_x$ does not exist? The key insight is that we can generalize the notion of gradient by considering *any* linear function that still provides a lower bound for f. We call such a function a **subgradient**.

Formally, a subgradient $\overline{\nabla} f|_x$ at point x is any vector satisfying:

$$f(x') \geq f(x) + \overline{\nabla} f|_x \cdot (x' - x) \quad \text{for all } x' \tag{3.29}$$

The crucial difference is that at non-differentiable points, there may be multiple valid subgradients—indeed, there is typically an entire set of them. Any vector whose corresponding linear function stays below the graph of f qualifies as a subgradient. At differentiable points, the subgradient coincides with the ordinary gradient, so this definition extends rather than contradicts the classical notion.

For the hinge loss specifically, we can observe geometrically what the valid subgradients are. At the corner point $ty = 1$, the function transitions from having slope $-t$ (when $ty < 1$) to having slope 0 (when $ty > 1$). Any line whose slope lies between these two values—that is, any slope in the interval $[-t, 0]$ when $t = 1$, or equivalently $[0, -t]$ when $t = -1$—will provide a valid lower bound and thus constitute a valid subgradient.



In practice, when implementing gradient descent with hinge loss, we typically choose a convenient subgradient from this range. A common choice is to use the horizontal line (slope 0) as the subgradient at $ty = 1$. This corresponds to assuming that $\frac{\partial L_H}{\partial y} = 0$ at the singular point, which gives us:

$$\overline{\nabla}_y L_H = \begin{cases} -t & ty < 1 \\ 0 & ty \geq 1 \end{cases} \tag{3.30}$$

This subgradient can be used in place of the ordinary gradient in gradient descent algorithms, and the resulting optimization procedure is guaranteed to converge to a minimum (though possibly more slowly than with smooth functions). The use of subgradients extends the applicability of gradient-based methods to a much broader class of convex but non-smooth loss functions, making practical optimization feasible even when classical calculus-based methods would fail.

Square loss in classification

Having examined loss functions specifically designed for classification, we might wonder: could we simply adapt regression losses to the classification setting? The **squared loss for classification** does exactly this, modifying the quadratic loss we saw earlier to account for the structure of binary classification:

$$L(t, y) = (1 - ty)^2 \tag{3.31}$$

Notice how this differs from the regression version $(t - y)^2$. Here, we penalize the squared difference between the desired quantity $ty = 1$ (which would indicate a correct and confident prediction) and the actual product ty of target and prediction.

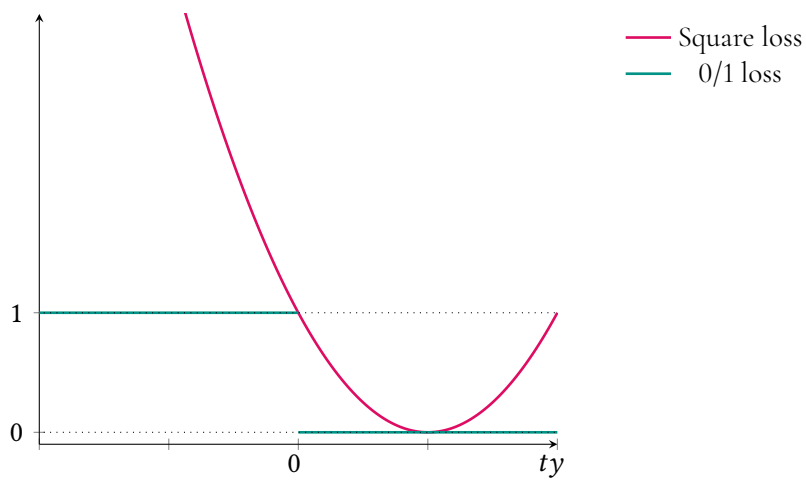
The squared loss for classification has some appealing mathematical properties:

- It is continuous everywhere and infinitely differentiable—as smooth as a function can be

- Its gradient is continuous and can be computed analytically everywhere
- It is convex, ensuring unique global minima
- Unlike hinge loss, there are no corners or non-differentiable points to worry about

However, these mathematical conveniences come with significant practical drawbacks:

- It is **not** a surrogate loss—it does not upper bound 0/1 loss everywhere. For correctly classified examples with $ty > 1$, the squared loss can be less than the 0/1 loss.
- **Extreme errors are heavily penalized**: Because errors are squared, a prediction that is very wrong (large negative ty) incurs an enormous penalty that grows quadratically. This makes the loss extremely sensitive to outliers or mislabeled examples in the training data.
- **Symmetric penalization around zero**: Even largely correct predictions with $ty \gg 1$ are penalized quadratically as ty increases. This seems counterintuitive—we are punishing the classifier for being very confident in correct predictions! This symmetry is inherited from the regression setting where over-prediction and under-prediction are both undesirable, but in classification, high confidence in correct predictions should generally be encouraged, not penalized.



Due to these drawbacks, squared loss is rarely the first choice for classification tasks, despite its mathematical simplicity. The other surrogate losses we have discussed typically provide better behavior in practice.

Log loss (cross entropy)

One of the most important and widely used loss functions in classification is **log loss**, also known as **cross entropy loss** or **logistic loss**. It plays a central role in logistic regression and serves as the standard loss function for probabilistic classification. The log loss is defined as:

$$L(t, y) = \frac{1}{\log 2} \log(1 + e^{-ty}) \tag{3.32}$$

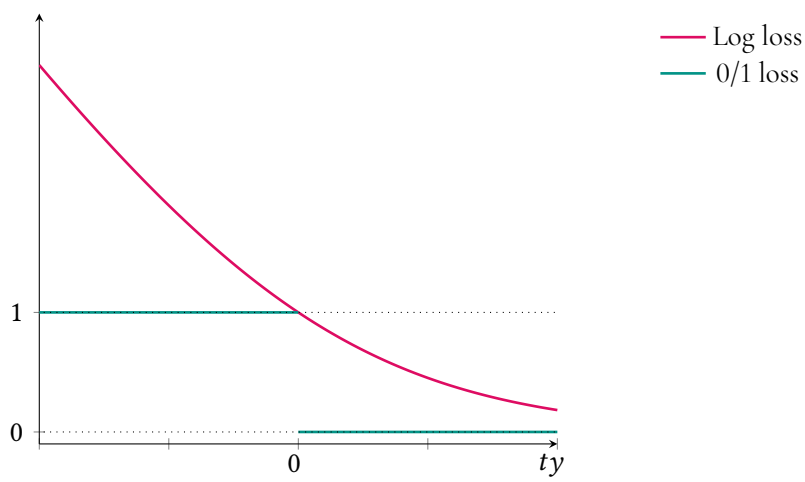
The constant $\frac{1}{\log 2}$ is a normalization factor (converting from natural logarithm to base-2 logarithm) and is sometimes omitted in practice, as it merely scales the loss by a constant factor without affecting the optimization.

The log loss can be understood as a smoothed version of hinge loss. Where hinge loss has a sharp corner at $ty = 1$, log loss curves smoothly through this region. As ty becomes large and negative (confident incorrect predictions), log loss grows approximately linearly, similar to hinge loss. As ty becomes large and positive (confident correct predictions), log loss approaches zero asymptotically but never quite reaches it—there is always a small penalty encouraging even more confidence.

The log loss possesses excellent mathematical properties:

- It is continuous and infinitely differentiable everywhere—perfectly smooth with no corners
- Its gradient is continuous and well-defined at all points, facilitating efficient gradient-based optimization
- It is strictly convex, ensuring a unique global minimum
- It is a proper surrogate loss, upper bounding 0/1 loss for all values of ty

One potential concern is that log loss can heavily penalize extremely wrong predictions. Since the loss grows without bound as $ty \to -\infty$, a single catastrophically mispredicted example (perhaps due to mislabeling or an outlier) can dominate the total loss. However, this growth is logarithmic rather than polynomial, making it more robust than squared loss while still encouraging the correction of serious errors.



Connection to information theory: cross entropy and KL divergence

The name "cross entropy" is not arbitrary—log loss has deep connections to information theory and probability theory that provide additional insight into why it is such a natural choice for probabilistic classification.

In information theory, given two probability distributions p and q over the same space, the **cross entropy** of q with respect to p measures the average number of bits needed to encode data distributed according to p when using a code optimized for distribution q. It is defined as:

$$\mathbb{H}(p, q) = -\mathop{\mathbb{E}}_{x \sim p}[\log q(x)] = -\int p(x) \log q(x) dx \tag{3.33}$$

This quantity is intimately related to the **Kullback-Leibler divergence**, which measures how different two probability distributions are:

$$D_{KL}(p||q) = -\int p(x)\log\frac{q(x)}{p(x)}dx = -\int p(x)\log q(x)dx + \int p(x)\log p(x)dx = \mathbb{H}(p,q) - \mathbb{H}(p) \quad (3.34)$$

where $\mathbb{H}(p) = -\mathbb{E}_{x\sim p}[\log p(x)]$ is the **entropy** of distribution p, representing the minimum average number of bits needed to encode symbols drawn from p.

To understand these concepts intuitively:

- The entropy $\mathbb{H}(p)$ represents the fundamental uncertainty or information content in distribution p—the minimum number of bits per symbol needed in a compression scheme when the true distribution is known.
- The cross entropy $\mathbb{H}(p,q)$ represents the average code length when we use a code optimized for distribution q but the true distribution is actually p. If q matches p perfectly, this equals the entropy; if they differ, we need more bits.
- The KL divergence $D_{KL}(p||q)$ measures the additional bits wasted on average by using q instead of the correct distribution p. It is always non-negative and equals zero if and only if $p = q$ almost everywhere.

Now, how does this connect to classification? Consider a classifier that outputs probabilities for each class. For binary classification:

- Let $p(\mathbf{x}) \in \{0,1\}$ be the true probability that input $\mathbf{x}$ belongs to class C_1. In the training set, this is simply the target label $t \in \{0,1\}$.
- Let $y(\mathbf{x}) \in [0,1]$ be the predicted probability that $\mathbf{x}$ belongs to class C_1.

The cross entropy between the true and predicted probability distributions, estimated empirically over the training set, is:

$$\mathrm{CE}(\mathcal{T}) = -\frac{1}{|\mathcal{T}|}\sum_{(\mathbf{x},t)\in\mathcal{T}}\Big(t\log y(\mathbf{x}) + (1-t)\log(1-y(\mathbf{x}))\Big) \quad (3.35)$$

This formula has an elegant interpretation: for each example, we compute the log probability that our classifier assigns to the correct class, then average these over all examples. The negative sign converts this into a loss (since high probability corresponds to low loss).

We can rewrite this by separating the sum based on which class each example belongs to:

$$\mathrm{CE}(\mathcal{T}) = -\frac{1}{|\mathcal{T}|}\Big(\sum_{(\mathbf{x},t)\in C_1}\log y(\mathbf{x}) + \sum_{(\mathbf{x},t)\in C_0}\log(1-y(\mathbf{x}))\Big) \quad (3.36)$$

Now suppose our classifier is a **logistic regression** model, where the predicted probability is given by applying the logistic (sigmoid) function to a linear combination of features:

$$y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}} \tag{3.37}$$

Substituting this into the cross entropy formula and using the algebraic identity $\log(\sigma(z)) = -\log(1 + e^{-z})$ and $\log(1 - \sigma(z)) = -\log(1 + e^{z})$, we obtain:

$$\mathrm{CE}(\mathcal{T}) = \frac{1}{|\mathcal{T}|}\Big(\sum_{(\mathbf{x},t)\in C_1} \log(1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}) + \sum_{(\mathbf{x},t)\in C_0} \log(1 + e^{\mathbf{w}^T\mathbf{x}+b})\Big) \tag{3.38}$$

Finally, if we encode our target labels as $t \in \{-1, +1\}$ rather than $\{0, 1\}$ (with $t = -1$ for class C_0 and $t = +1$ for class C_1), we can unify both cases into a single formula:

$$\mathrm{CE}(\mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \log(1 + e^{-t(\mathbf{w}^T\mathbf{x}+b)}) \tag{3.39}$$

This is precisely the empirical risk when we use log loss (up to the constant factor $\log 2$):

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|\log 2} \sum_{(\mathbf{x},t)\in\mathcal{T}} \log(1 + e^{-t(\mathbf{w}^T\mathbf{x}+b)}) \tag{3.40}$$

This beautiful connection reveals that minimizing log loss is equivalent to minimizing the cross entropy between the true class labels and our predicted probabilities—that is, to finding the model whose predictions are closest to the true distribution in an information-theoretic sense. This provides strong theoretical justification for the use of log loss in probabilistic classification.

Exponential loss

Our final surrogate loss function is the **exponential loss**, which is defined as:

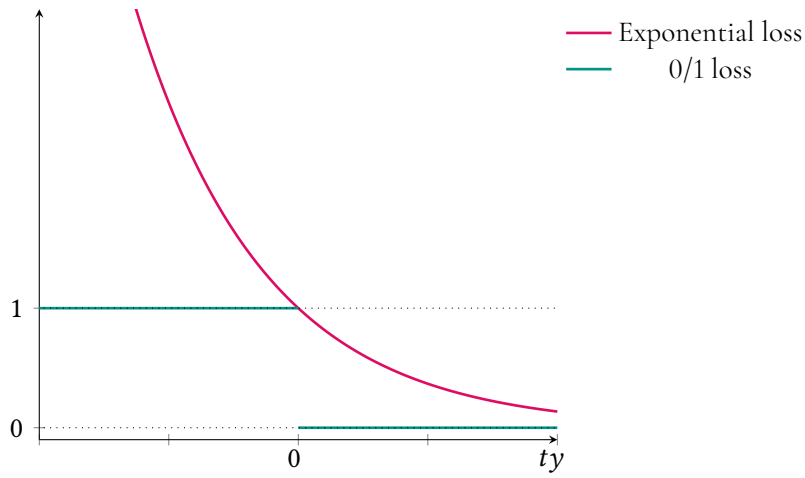
$$L(t, y) = e^{-ty} \tag{3.41}$$

The exponential loss shares many properties with log loss but differs in how aggressively it penalizes errors:

- It is used prominently in boosting algorithms, particularly AdaBoost, where it provides a natural framework for combining weak learners
- It penalizes wrong predictions more severely than log loss does—the penalty grows exponentially rather than logarithmically as errors become larger
- It is continuous and infinitely differentiable everywhere, with a continuous gradient at all points

- It is strictly convex, ensuring a unique global minimum
- It is a proper surrogate loss, upper bounding 0/1 loss

The key distinguishing feature of exponential loss is its aggressive treatment of errors. For large negative values of ty (confident incorrect predictions), the exponential loss grows extremely rapidly—much faster than log loss. This makes exponential loss very sensitive to outliers and mislabeled data. On the other hand, this aggressive penalization can be beneficial in the boosting framework, where we explicitly want to focus attention on examples that previous classifiers have struggled with.



The choice between exponential loss, log loss, and other surrogates often depends on the specific characteristics of the problem at hand: the expected prevalence of outliers, the relative importance of different types of errors, and the broader algorithmic context in which the loss function will be used. Each surrogate encodes different inductive biases about what makes a good classifier, and understanding these differences is crucial for selecting the right tool for a given task.

Computing h^*

The problem of computing the optimal hypothesis h^* typically amounts to solving a risk minimization problem over a suitable hypothesis space. In many practical learning scenarios, the hypothesis class is parameterized by a real-valued vector $\theta \in \Theta$, where $\Theta = \mathbb{R}^d$ for some $d > 0$. Under this assumption, the minimization of the empirical risk $\overline{\mathcal{R}}_{\mathcal{T}}(h_\theta)$ becomes an unconstrained optimization problem.

In this setting, a necessary condition for optimality is that θ corresponds to a (local) stationary point of the empirical risk. This can be found by setting all partial derivatives of $\overline{\mathcal{R}}_{\mathcal{T}}(h_\theta)$ with respect to the parameters to zero:

$$\frac{\partial}{\partial \theta_i} \overline{\mathcal{R}}_{\mathcal{T}}(h_\theta) = 0. \tag{3.42}$$

Equivalently, this condition can be expressed in vector form by requiring the gradient of the empirical risk to vanish:

$$\nabla \overline{\mathcal{R}}_{\mathcal{T}}(h_\theta) = \mathbf{0}. \tag{3.43}$$

Although conceptually simple, the analytical solution of this system of equations is in general difficult or even infeasible, especially when the hypothesis function is complex or when the loss function is non-linear. As a consequence, numerical optimization techniques are almost always employed in practice in order to approximately compute a minimizer of the empirical risk.

Gradient descent

Gradient descent is one of the most widely used numerical methods for minimizing a differentiable function $J(\theta)$. The method proceeds iteratively, starting from an initial guess $\theta^{(0)}$, and updates the parameters by moving them in the direction opposite to the gradient of the objective function, which locally corresponds to the direction of steepest ascent.

At iteration k, the update rule is given by

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla J|_{\theta^{(k)}}$$

where $\nabla J|_{\theta^{(k)}}$ denotes the gradient of J evaluated at the current parameter vector $\theta^{(k)}$.

Component-wise, the update can be written as

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta \frac{\partial J(\theta)}{\partial \theta_j}\bigg|_{\theta^{(k)}}. \tag{3.44}$$

It is convenient to rewrite this update in terms of the increment $\Delta_{j,k}$ of θ_j as a sequence of four steps:

$$\begin{aligned} g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \\ \eta_{j,k} &= \eta \\ \Delta_{j,k} &= -\eta_{j,k}\, g_{j,k} \\ \theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k} \end{aligned}$$

The scalar $\eta_{j,k}$ is known as the *learning rate* and controls the magnitude of the update at each step and direction. Its choice is critical, as it directly influences both the speed of convergence and the stability of the optimization process. Observe that, even if we take into account, for generality, that η could assume different values $\eta_{j,k}$ at different steps (k) and for different parameters (j), in the case of basic gradient descent considered the learning rate is assumed constant for all parameters and along the descent.

Batch gradient descent

When gradient descent is applied to the minimization of the empirical risk, the objective function coincides with the average loss over the entire training set $\mathcal{T}$. In this case, the function $J(\theta)$ takes the form of the empirical risk

$$J(\theta) = \overline{\mathcal{R}}_{\mathcal{T}}(h_\theta) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h_\theta(\mathbf{x}), t) \tag{3.45}$$

As a consequence,

$$g_{j,k} = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \tag{3.46}$$

Overall, the update rule for θ_j takes the form

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \tag{3.47}$$

As a concrete example, consider linear regression, where the hypothesis function is defined as

$$h(\mathbf{x}) = \sum_{s=1}^{d} \theta_s x_s + \theta_0, \tag{3.48}$$

and the loss function is the squared error

$$L(h(\mathbf{x}), t) = (h(\mathbf{x}) - t)^2 = \left(\sum_{s=1}^{d} \theta_s x_s + \theta_0 - t \right)^2. \tag{3.49}$$

The corresponding partial derivatives are

$$\begin{aligned} \frac{\partial}{\partial \theta_j} L(h_\theta(\mathbf{x}), t) &= \left(\sum_{s=1}^{d} \theta_s x_s + \theta_0 - t \right) x_j \qquad & j = 1, \dots, d \\ \frac{\partial}{\partial \theta_0} L(h_\theta(\mathbf{x}), t) &= \left(\sum_{s=1}^{d} \theta_s x_s + \theta_0 - t \right) \end{aligned}$$

Substituting these expressions into the update rule yields

$$\begin{aligned} \theta_j^{(k+1)} &= \theta_j^{(k)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right) x_j \qquad & j = 1, \dots, d \\ \theta_0^{(k+1)} &= \theta_0^{(k)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right) \end{aligned}$$

This procedure is referred to as **batch gradient descent**, since at each iteration the gradient is computed using the entire training set. While this guarantees a smooth and stable decrease of the empirical risk, it also implies

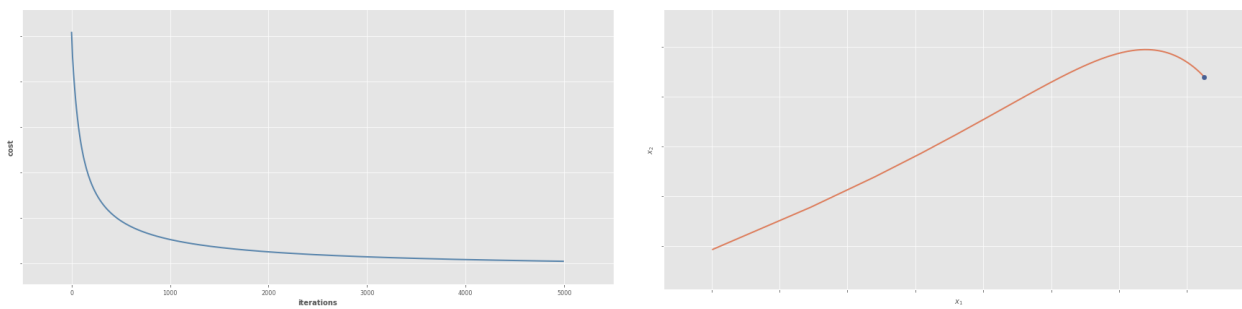
that each update can be computationally expensive. In particular, batch gradient descent can be prohibitively slow for large datasets or infeasible when the dataset does not fit entirely in memory. Moreover, it does not naturally support online learning scenarios, where data arrive sequentially.

In pseudocode, batch gradient descent can be summarized as follows:

```
for i in range(nb_epochs):
  params_grad = evaluate_gradient(loss_function, data, params)
  params = params - learning_rate * params_grad
```

For each epoch, the gradient of the loss function with respect to the parameters is computed over the whole dataset. Modern deep learning libraries typically rely on automatic differentiation techniques to efficiently perform this computation.

Batch gradient descent is guaranteed to converge to the global minimum when the objective function is convex, and to a local minimum in the non-convex case.



(a) Cost vs number of iterations

(b) Trajectory in feature space

Figure 3.7. Batch gradient descent behavior

Stochastic gradient descent

A major limitation of batch gradient descent is the need to recompute gradients over the entire dataset before each parameter update. This redundancy can lead to excessive computational costs, especially for large-scale learning problems. Stochastic gradient descent (SGD) addresses this issue by performing updates using a single training example $(\mathbf{x}, t)$ at a time.

This correspond to consider a different function to minimize, in dependance from the example considered

$$J(\theta) = L(h_\theta(\mathbf{x}), t) \tag{3.50}$$

As a consequence, the gradient function itself is different at each step, depending from the traning example currently considered

$$g_{j,k} = \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \tag{3.51}$$

At each iteration the parameters are updated based on the gradient of the loss evaluated at the selected training example $(\mathbf{x}, t)$:

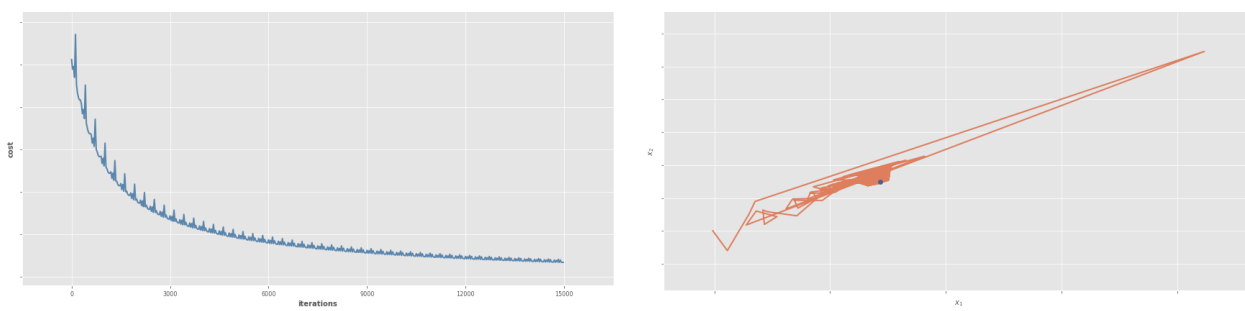
$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta \frac{\partial}{\partial \theta_j} L(h_\theta(\mathbf{x}), t) \Big|_{\theta^{(k)}} \tag{3.52}$$

Since updates are performed much more frequently, SGD is typically significantly faster than batch gradient descent and naturally supports online learning. However, the price to pay is a much higher variance in the updates, due to the changing function to be minimized and gradient, which causes the values returned to fluctuate considerably, as illustrated in Figure 1.8. So, while batch gradient descent follows a smooth trajectory toward a minimum, SGD exhibits a noisy and erratic path, with occasional increases in the loss.

This stochasticity can be beneficial, as it allows the algorithm to escape shallow local minima or saddle points. On the other hand, it can make convergence to an exact minimum more difficult. However, it can be shown that by gradually decreasing the learning rate over time, SGD converges almost surely to a local minimum, recovering the asymptotic behavior of batch gradient descent.

A typical implementation of SGD is shown below:

```
for i in range(nb_epochs):
  np.random.shuffle(data)
  for example in data:
    params_grad = evaluate_gradient(loss_function, example, params)
    params = params - learning_rate * params_grad
```



(a) Cost vs number of iterations

(b) Trajectory in feature space

Figure 3.8. Stochastic gradient descent behavior

In the case of linear regression, the SGD updates reduce to

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right) x_j \qquad j = 1, \dots, d$$

$$\theta_0^{(k+1)} = \theta_0^{(k)} - \eta \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right)$$

Mini-batch gradient descent

Mini-batch gradient descent represents an intermediate strategy between batch gradient descent and SGD. The idea is that the dataset is partitionated into a collection of subsets (mini-batches) each of a predefined size m. During each epoch, the method iterates on all mini-batches and, at each iteration, the gradient is computed using the current mini-batch B of the training set, that is with respect to the empirical risk restricted to examples in B_r

$$J(\theta) = \frac{1}{m} \sum_{(\mathbf{x},t) \in B} L(h_\theta(\mathbf{x}), t) \tag{3.53}$$

the resulting gradient is

$$g_{j,k} = \frac{1}{m} \sum_{(\mathbf{x},t) \in B} \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j} \Big|_{\theta^{(k)}} \tag{3.54}$$

The update rule for θ_j is then

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \frac{\eta}{m} \sum_{(\mathbf{x},t) \in B} \frac{\partial L(h_\theta(\mathbf{x}), t)}{\partial \theta_j} \Big|_{\theta^{(k)}}. \tag{3.55}$$

Mini-batch gradient descent combines the advantages of the two previous approaches:

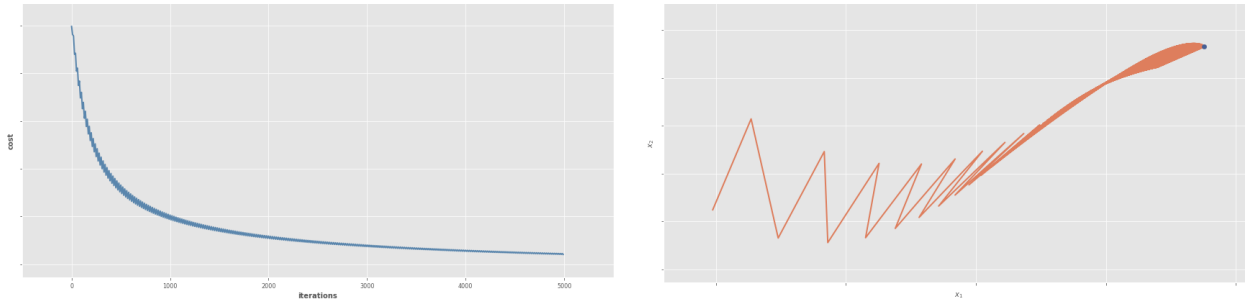
- it limits the computational cost of each update by avoiding full passes over the dataset
- it reduces the variance of parameter updates compared to SGD, leading to more stable convergence, moreover the computation of a fixed number of gradients can be parallelized on GPUs, making it possible to speed up the whole precess.

The mini-batch size m is itself a hyperparameter, with typical values ranging from 32 to 256, depending on the problem and the available computational resources. In practice, mini-batch gradient descent is the standard choice for training neural networks, and the term *SGD* is often used to refer to this variant as well.

A typical implementation is as follows:

```
for i in range(nb_epochs):
  np.random.shuffle(data)
  for batch in get_batches(data, batch_size=m):
```

```
params_grad = evaluate_gradient(loss_function, batch, params)
params = params - learning_rate * params_grad
```



(a) Cost vs number of iterations

(b) Trajectory in feature space

Figure 3.9. Mini-batch gradient descent behavior

For linear regression, the corresponding update rules are

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \frac{\eta}{m} \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right) x_j, \qquad j = 1, \dots, d$$

$$\theta_0^{(k+1)} = \theta_0^{(k)} - \frac{\eta}{m} \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_s^{(k)} x_s + \theta_0^{(k)} - t \right).$$

Open issues

The optimization strategies discussed so far mainly differ in the number of training examples used to estimate the gradient before performing a parameter update. Despite their widespread use, these methods present several challenges that can hinder effective convergence:

- Selecting an appropriate learning rate η is non-trivial. If η is too small, convergence can be extremely slow; if it is too large, the optimization process may become unstable and even diverge.
- To mitigate this issue, learning rate schedules or adaptive reduction strategies can be employed, for instance by decreasing η over time or when the decrease in loss becomes negligible. However, such mechanisms rely on hyperparameters that must be specified in advance and may not adapt optimally to different datasets.
- A single global learning rate is applied to all parameters. This can be suboptimal when features have different scales or frequencies, as it may be desirable to update some parameters more aggressively than others.
- Many practical optimization problems, especially in neural networks and deep learning, are highly non-convex and characterized by numerous local minima and saddle points. In particular, saddle points are often surrounded by flat regions where gradients are close to zero in all directions, making it difficult for basic gradient-based methods to escape.

Momentum gradient descent

Momentum gradient descent is motivated by a physical interpretation of the optimization process, in which parameter updates are modeled as the motion of a point-like body sliding over the surface defined by the cost function $J(\theta)$. In this analogy, the parameter vector θ represents the position of a body of unit mass $m = 1$, subject to a force induced by the shape of the cost surface.

The force acting on the body is assumed to be conservative and derived from a potential energy function $U(\theta) = \eta\, J(\theta)$, so that

$$F(\theta) = -\nabla U(\theta) = -\eta\, \nabla J(\theta). \tag{3.56}$$

Under this assumption, the negative gradient $-\eta\, \nabla J(\theta)$ plays the role of the force—and, since the mass is unitary, also of the acceleration—applied to the body at position θ. The constant η therefore corresponds to the gravitational constant g in the classical expression of the weight force $F = -mgh$.

In standard gradient descent, the movement of the body is entirely determined by the instantaneous acceleration at the current position. This corresponds to directly updating the parameters according to

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta_{j,k}\, g_{j,k} = \theta_j^{(k)} - \eta_{j,k} \frac{\partial J(\theta)}{\partial \theta_j}\bigg|_{\theta^{(k)}} \tag{3.57}$$

implicitly assuming that the velocity is reset at each step and that the system has no memory of past movements.

In **momentum gradient descent**, a more faithful physical model is adopted. The update of the parameters is governed by the velocity of the body, rather than directly by the acceleration. The position is updated as

$$\theta_j^{(k+1)} = \theta_j^{(k)} + v_j^{(k+1)} \tag{3.58}$$

while the velocity evolves according to the applied acceleration:

$$v_j^{(k+1)} = v_j^{(k)} - \eta_{j,k}\, g_{j,k} \tag{3.59}$$

In this way, the current update depends not only on the present gradient, but also on the cumulated effect of past gradients through the velocity term. which results into

$$\theta_j^{(k+1)} = \theta_j^{(0)} - \sum_{r=0}^{k} \eta_{j,r}\, g_{j,r} + v_j^{(0)} \tag{3.60}$$

Assuming $v_j^{(0)} = 0$ and the learning rate constant for all parameters and at each step, we obtain the simpler update rule

$$\theta_j^{(k+1)} = \theta_j^{(0)} - \eta \sum_{r=0}^{k} g_{j,r} \tag{3.61}$$

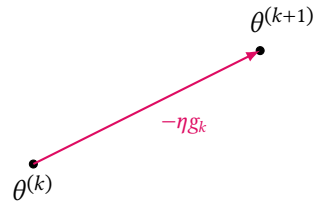


Figure 3.10. Gradient descent

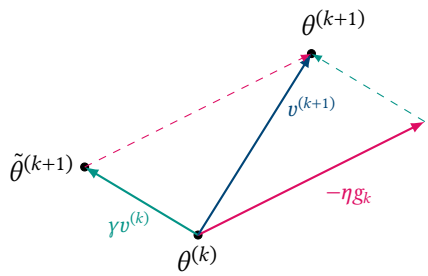


Figure 3.11. Momentum gradient descent

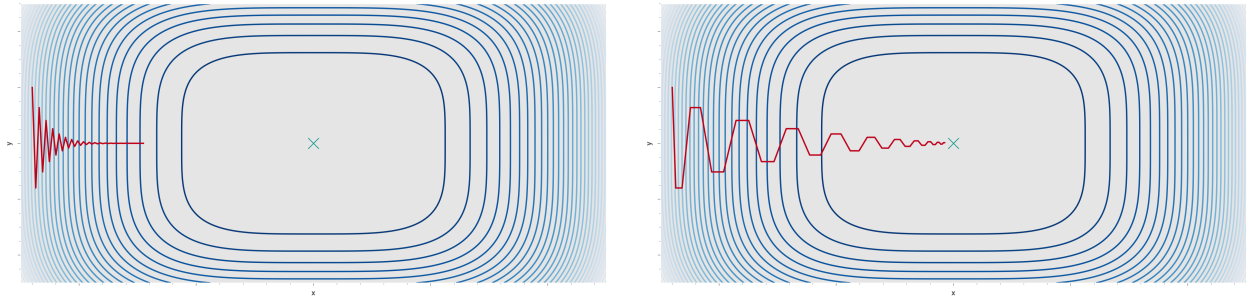
$$\theta_j^{(k+1)} = \theta_j^{(0)} - \eta \sum_{r=0}^{k} g_{j,r} \tag{3.62}$$

This formulation makes explicit that momentum gradient descent effectively updates the parameters using a weighted sum of past gradients, which corresponds, in physical terms, to integrating past accelerations. As a result, the velocity tends to increase along directions where gradients are consistently aligned, while oscillations along directions with rapidly changing gradients are dampened.

This behavior explains why momentum helps mitigate oscillations in narrow valleys or ravines of the loss surface. As illustrated in Figure 1.12, the presence of momentum significantly reduces fluctuations in directions orthogonal to the main descent direction, allowing faster and more stable progress toward a minimum. In contrast, standard gradient descent tends to oscillate across steep directions, slowing down convergence.

In practice, momentum gradient descent uses a dampened sum of previous gradients by introducing an additional hyperparameter $\gamma \in [0, 1)$, which controls how much of the previous velocity is retained at each step. In the physical analogy, this corresponds to introducing friction or attrition.

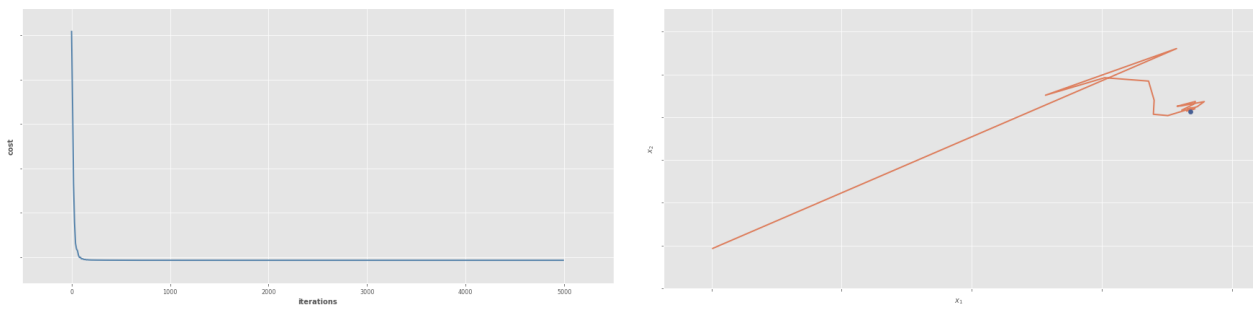
$$\begin{aligned} v_j^{(k+1)} &= \gamma v_j^{(k)} - \eta g_{j,k} \\ \theta_j^{(k+1)} &= \theta_j^{(k)} + v_j^{(k+1)} = \theta_j^{(0)} - \eta \sum_{r=1}^{k} \gamma^{k-r} g_{j,r} \end{aligned}$$



(a) GD without momentum

(b) Momentum GD

Figure 3.12. Momentum effect on trajectory



(a) Cost vs number of iterations

(b) Trajectory in feature space

Figure 3.13. Momentum gradient descent behavior

For linear regression, this yields the following component-wise updates, under the mini-batch approach:

$$
\begin{aligned}
v_j^{(k+1)} &= \gamma v_j^{(k)} - \eta \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_j^{(k)} x_s + \theta_0^{(k)} - t \right) x_j, \qquad j = 1, \dots, d \\
v_0^{(k+1)} &= \gamma v_0^{(k)} - \eta \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_j^{(k)} x_s + \theta_0^{(k)} - t \right)
\end{aligned}
$$

and

$$
\begin{aligned}
\theta_j^{(k+1)} &= \theta_j^{(k)} + v_j^{(k+1)} = \theta_j^{(0)} - \eta \sum_{r=1}^{k} \gamma^{k-r} \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_j^{(r)} x_s + \theta_0^{(r)} - t \right) x_j \qquad j = 1, \dots, d \\
\theta_0^{(k+1)} &= \theta_0^{(k)} + v_0^{(k+1)} = \theta_0^{(0)} - \eta \sum_{r=1}^{k} \gamma^{k-r} \sum_{(\mathbf{x},t)\in B} \left(\sum_{s=1}^{d} \theta_j^{(r)} x_s + \theta_0^{(r)} - t \right)
\end{aligned}
$$

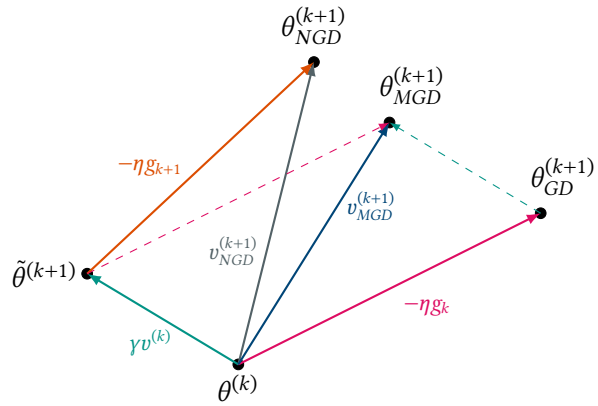


Figure 3.14. Nesterov vs momentum GD vs GD steps

Nesterov accelerated gradient descent

Momentum gradient descent implicitly assumes that the gradient evaluated at the current point $\theta^{(k)}$ provides sufficient information about the local behavior of the cost function. However, since momentum already moves the parameters in a predictable direction, it is possible to improve the update by anticipating this movement.

Specifically, adding the momentum term $\gamma v^{(k)}$ to $\theta^{(k)}$ defines a look-ahead estimate

$$\tilde{\theta}^{(k+1)} \triangleq \theta^{(k)} + \gamma v^{(k)}, \tag{3.63}$$

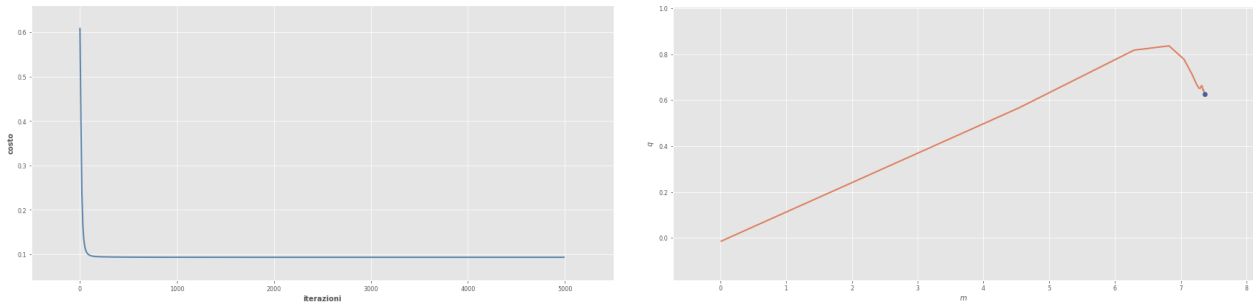
which approximates the next position of the parameters before the gradient is actually computed.

Nesterov accelerated gradient (NAG) exploits this idea by evaluating the gradient at the look-ahead point rather than at the current position. The update equations become

$$\begin{aligned}
\tilde{\theta}_j^{(k+1)} &= \theta_j^{(k)} + \gamma v_j^{(k)} \\
v_j^{(k+1)} &= \gamma v_j^{(k)} - \eta g_{j,k+1} \\
\theta_j^{(k+1)} &= \tilde{\theta}_j^{(k)} + v_j^{(k+1)}.
\end{aligned}$$

Compared to standard momentum, Nesterov accelerated gradient anticipates changes in the velocity and corrects the trajectory earlier, resulting in more responsive and often faster convergence. This behavior is summarized as follows:

- The same momentum-based update structure is preserved.
- The gradient is evaluated at an approximated future position rather than at the current one.
- Parameter and velocity updates are effectively anticipated with respect to momentum gradient descent.



(a) Cost vs number of iterations

(b) Trajectory in feature space

Figure 3.15. Nesterov accelerated gradient descent behavior

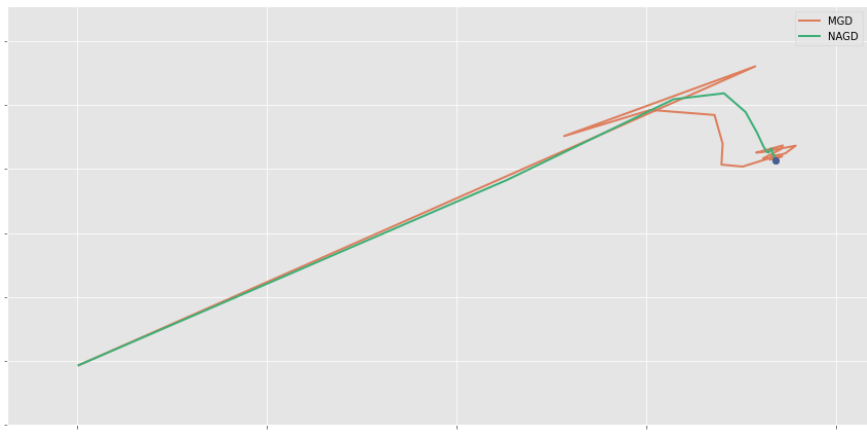


Figure 3.16. Comparison of MGD and NGD trajectories

Dynamically updating the learning rate

The learning rate η is a crucial hyperparameter in gradient-based optimization algorithms, as it directly controls the magnitude of each update step taken in the parameter space. Its choice strongly influences both the stability and the speed of convergence of the learning process. In particular:

- If η is too large, the updates may overshoot the local minimum, causing oscillations or even divergence, with a consequent increase of the loss function.
- If η is too small, the optimization process makes very slow progress, potentially getting trapped in flat regions or shallow local minima.

An appropriate learning rate should therefore enable steady and consistent progress toward a local minimum. However, using a single, fixed learning rate throughout the entire optimization process is often suboptimal. In the early stages of training, when the parameters are far from an optimum, larger steps are typically desirable in order to quickly reduce the loss. Conversely, when the algorithm approaches a local minimum, smaller steps are required to finely adjust the parameters and avoid oscillations around the optimum.

These contrasting requirements motivate the introduction of **learning rate schedules**, which progressively modify the learning rate during training according to a predefined rule. In this setting, η is updated at each iteration or epoch, allowing the optimization process to adapt its step size over time. Common examples include:

- **Step decay**, where the learning rate is reduced by a constant factor c every K steps (or epochs), i.e. every K epochs $\eta \leftarrow \frac{\eta}{c}$.
- **Exponential decay**, defined by $\eta^{(k)} = \eta^{(0)} e^{-\alpha k}$, which yields a smooth and continuous reduction of the learning rate.
- **$1/t$ decay**, given by $\eta^{(k)} = \frac{\eta^{(0)}}{1 + \alpha k}$, which decreases the learning rate more slowly at later stages.

While learning rate schedules are effective in practice, they suffer from two main limitations. First, their hyperparameters (such as c, K, or α) must be chosen in advance and are often highly problem-dependent. Second, the same learning rate is applied uniformly to all model parameters, regardless of their individual behavior during training.

A more refined strategy consists in assigning a distinct learning rate η_i to each parameter θ_i, and updating it dynamically based on the history of the gradients with respect to that parameter. In this way, parameters associated with consistently large gradients are assigned smaller learning rates, preventing excessively large updates, while parameters that change slowly over time are assigned larger learning rates, increasing their sensitivity to small but informative gradients. This idea underlies a family of *adaptive learning rate methods*, which we now discuss.

Adagrad

In standard gradient descent, the update rule for the parameter θ_j is

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \eta \frac{\partial J(\theta)}{\partial \theta_j}\Big|_{\theta^{(k)}}, \tag{3.64}$$

where the learning rate η is shared by all parameters.

Let us recall the definition of an update in terms of:

$$\begin{aligned} g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\Big|_{\theta^{(k)}} \\ \eta_{j,k} &= \eta \\ \Delta_{j,k} &= -\eta_{j,k} g_{j,k} \\ \theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k} \end{aligned}$$

Adagrad modifies the computation of $\Delta_{j,k}$ by assuming a parameter-specific learning rate, which adapts to the historical behavior of the gradients. The key idea is to perform larger updates for parameters that are rarely updated and smaller updates for parameters that frequently receive large gradients. This makes Adagrad particularly well-suited for problems with sparse data or highly heterogeneous features.

In Adagrad, each parameter θ_j is associated with its own learning rate η_j which is dinamically updated at each step: η_{jk} denotes its value after step k and is defined as

$$\eta_{j,k} = \frac{\eta}{\sqrt{G_{j,k} + \varepsilon}}, \tag{3.65}$$

where $G_{j,k}$ is the sum of the previous squared gradients

$$G_{j,k} = g_{j,0}^2 + g_{j,1}^2 + \cdots + g_{j,k}^2 = \sum_{i=0}^{k} g_{j,i}^2 \tag{3.66}$$

The starting learning rate η is then reduced after k steps by dividing it by the (rooted) sum of squared gradients with respect to θ_j up to iteration k: this can be seen as a proxy of the cumulative sum of the modules of all such gradients. The small constant ε is introduced for numerical stability, preventing division by zero.

The resulting update rule can be summarized as

$$\Delta_{j,k} = -\frac{\eta}{\sqrt{G_{j,k} + \varepsilon}} g_{j,k} \tag{3.67}$$

leading to the iterative scheme

$$\begin{aligned}
g_{j,k} &= \left.\frac{\partial J(\theta)}{\partial \theta_j}\right|_{\theta^{(k)}} \\
G_{j,k} &= G_{j,k-1} + g_{j,k}^2 = \sum_{i=0}^{k} g_{j,i}^2 \\
\eta_{j,k} &= \frac{\eta}{\sqrt{G_{j,k} + \varepsilon}} \\
\Delta_{j,k} &= -\eta_{j,k} g_{j,k} \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k}
\end{aligned}$$

with $G_{j,k} = 0$ for $k < 0$.

As training progresses, the denominator $\sqrt{G_{j,k} + \varepsilon}$ increases monotonically, causing the effective learning rate $\eta_{j,k}$ to decrease over time. Parameters that experienced large gradients in the past will see their learning rates decay more rapidly, effectively damping their updates, while parameters with many small past gradients retain relatively larger learning rates. Despite these advantages, the monotonic decay of the learning rate eventually leads to excessively small updates, effectively halting learning. This limitation motivates the development of improved adaptive methods where the effect of past gradients is reduced as new steps are performed.

RMSprop

RMSprop addresses the main drawback of Adagrad by preventing the learning rate from decaying too aggressively. Instead of accumulating all past squared gradients, RMSprop uses an exponentially decaying average, thereby limiting the influence of very old gradients.

Specifically, the accumulated squared gradient $G_{j,k}$ is replaced by its decaying counterpart $\tilde{G}_{j,k}$:

$$\begin{aligned}
\tilde{G}_{j,k} &= \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2 \\
&= \gamma^2 \tilde{G}_{j,k-2} + \gamma(1-\gamma) g_{j,k-1}^2 + (1-\gamma) g_{j,k}^2 \\
&= \gamma^3 \tilde{G}_{j,k-3} + \gamma^2(1-\gamma) g_{j,k-2}^2 + \gamma(1-\gamma) g_{j,k-1}^2 + (1-\gamma) g_{j,k}^2 \\
&= \gamma^k (1-\gamma) g_{j,0}^2 + \gamma^{k-1}(1-\gamma) g_{j,1}^2 + \gamma^{k-2}\gamma(1-\gamma) g_{j,2}^2 + \cdots + (1-\gamma) g_{j,k}^2 \\
&= (1-\gamma) \sum_{i=0}^{k} \gamma^{k-i} g_{j,i}^2
\end{aligned}$$

where $0 < \gamma < 1$ controls at each step how much the new cumulative gradient $\tilde{G}_{jk}$ is affected by the current gradient $g_{j,k}$, and can be seen as the decay rate in the decaying cumulative gradient. That is, the value considered in $\tilde{G}_{j,k}$ for a gradient g reduces by γ at each new step. is It is assumed that $\tilde{G}_{j,k} = 0$ for $k < 0$.

The parameter-specific learning rate is then defined as

$$\eta_{j,k} = \frac{\eta}{\sqrt{\tilde{G}_{j,k} + \varepsilon}}, \tag{3.68}$$

leading to the update rule

$$\begin{aligned}
g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\Big|_{\theta^{(k)}}, \\
\tilde{G}_{j,k} &= \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2 = (1-\gamma) \sum_{i=0}^{k} \gamma^{k-i} g_{j,i}^2 \\
\eta_{j,k} &= \frac{\eta}{\sqrt{\tilde{G}_{j,k} + \varepsilon}} \\
\Delta_{j,k} &= -\eta_{j,k} g_{j,k} \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k}.
\end{aligned}$$

This can be seen as approximately reducing at each step the learning rate η by the cumulative sum of the modules of past gradients, reduced in such a way that the effect of older gradients is reduced by exponentially discounting them. In this way RMSprop maintains a learning rate that mainly adapts to recent gradient magnitudes, allowing continued learning even in long training runs. Typical values are $\gamma \simeq 0.9$, $\eta \simeq 0.1$, and $\varepsilon \simeq 10^{-8}$.

Adadelta

Adadelta further refines RMSprop by eliminating the need to manually specify a value for the global learning rate η. Instead, it rescales parameter updates using a decaying average of past squared updates.

Let

$$
\begin{aligned}
\overline{G}_{j,k} &= \gamma \overline{G}_{j,k-1} + (1-\gamma)\Delta_{j,k}^2 \\
&= \gamma^k (1-\gamma)\Delta_{j,0}^2 + \gamma^{k-1}(1-\gamma)\Delta_{j,1}^2 + \gamma^{k-2}\gamma(1-\gamma)\Delta_{j,2}^2 + \cdots + (1-\gamma)\Delta_{j,k}^2 \\
&= (1-\gamma)\sum_{i=0}^{k} \gamma^{k-i}\Delta_{j,i}^2,
\end{aligned}
$$

with $\overline{G}_{j,k} = 0$ for $k < 0$.

The Adadelta update rule is then

$$
\begin{aligned}
g_{j,k} &= \frac{\partial J(\theta)}{\partial \theta_j}\Big|_{\theta^{(k)}}, \\
\tilde{G}_{j,k} &= \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2 = (1-\gamma)\sum_{i=0}^{k} \gamma^{k-i} g_{j,i}^2 \\
\eta_{j,k} &= -\frac{\sqrt{\overline{G}_{j,k-1} + \varepsilon}}{\sqrt{\tilde{G}_{j,k} + \varepsilon}} \\
\Delta_{j,k} &= -\eta_{j,k} g_{j,k} \\
\overline{G}_{j,k} &= \gamma \overline{G}_{j,k-1} + (1-\gamma)\Delta_{j,k}^2 = (1-\gamma)\sum_{i=0}^{k} \gamma^{k-i}\Delta_{j,i}^2 \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k}.
\end{aligned}
$$

This formulation ensures that the units of the update $\Delta_{j,k}$ are consistent with those of the parameter θ_j. Typical values are $\gamma \simeq 0.95$ and $\varepsilon \simeq 10^{-8}$.

Adam

Adam (Adaptive Moment Estimation) combines the advantages of Adagrad and RMSprop and momentum methods by introducing, in addition to $\tilde{G}_{j,k}$ a momentum term as an exponentially decaying sums of gradients

$$
\tilde{H}_{j,k} = \beta \tilde{H}_{j,k-1} + (1-\beta) g_{j,k} = (1-\beta)\sum_{i=0}^{k} \beta^{k-i} g_{j,i} \tag{3.69}
$$

Both terms are initialized as $\tilde{G}_{j,k} = \tilde{H}_{j,k} = 0$: because of this initialization, during the initial steps of training, these estimates are biased towards zero. This is especially problematic when the decay rates β and γ are are close to 1 (which they usually are, e.g., 0.9 and 0.999). Without correction, the terms in the sequence would remain very small for a long time, significantly slowing down the learning process or making it unstable.

Bias correction solves this by dividing the moving averages by a factor that compensates for the initialization.

$$\hat{G}_{j,k} = \frac{\tilde{G}_{j,k}}{1-\gamma^k}$$
$$\hat{H}_{j,k} = \frac{\tilde{H}_{j,k}}{1-\beta^k}.$$

The effect of these corrections is that $\hat{G}_{j,1}, \hat{G}_{j,2}, \hat{G}_{j,3}, \ldots$ are computed by scaling the original values $\tilde{G}_{j,1}$, $\tilde{G}_{j,2}$, $\tilde{G}_{j,3}, \ldots$ by $1-\gamma < 1-\gamma^2 < 1-\gamma^3 < \ldots$ which are smaller for the first values, the most dependent from the initial one. This corresponds to scaling up the first values $\tilde{G}_{j,k}$, which tend to be small as an effect of the initialization $\tilde{G}_{j,0} = 0$. The same considerations hold for $\hat{H}_{j,1}, \hat{H}_{j,2}, \ldots$.

In other terms, as the number of iterations k increases, the denominators $1-\beta^k$ and $1-\gamma^k$ approach 1, meaning the correction becomes less significant as the algorithm gains more "confidence" from the observed data.

The complete update rule is therefore

$$\begin{aligned}
g_{j,k} &= \left.\frac{\partial J(\theta)}{\partial \theta_j}\right|_{\theta^{(k)}} \\
\tilde{G}_{j,k} &= \gamma \tilde{G}_{j,k-1} + (1-\gamma) g_{j,k}^2 = (1-\gamma) \sum_{i=0}^{k} \gamma^{k-i} g_{j,i}^2 \\
\tilde{H}_{j,k} &= \beta \tilde{H}_{j,k-1} + (1-\beta) g_{j,k} = (1-\beta) \sum_{i=0}^{k} \beta^{k-i} g_{j,i} \\
\hat{G}_{j,k} &= \frac{\tilde{G}_{j,k}}{1-\gamma^k} \\
\hat{H}_{j,k} &= \frac{\tilde{H}_{j,k}}{1-\beta^k} \\
\eta_{j,k} &= \frac{\eta}{\sqrt{\hat{G}_{j,k} + \varepsilon}} \\
\Delta_{j,k} &= -\eta_{j,k} \hat{H}_{j,k} \\
\theta_j^{(k+1)} &= \theta_j^{(k)} + \Delta_{j,k}.
\end{aligned}$$

Adam is widely used in practice due to its robustness and good empirical performance. Standard default values are $\eta \simeq 0.001$, $\gamma \simeq 0.9$, $\beta \simeq 0.999$, and $\varepsilon \simeq 10^{-8}$.

Reassuming table

Method	$G^{(k)}$	$\eta^{(k)}$
GD	$g^{(k)}$	η
Adagrad	$g^{(k)}$	$\frac{\eta}{RSS(g^2)}$
RMSprop	$g^{(k)}$	$\frac{\eta}{RDSS(g^2,\gamma)}$
Adadelta	$g^{(k)}$	$\frac{RDSS(\Delta^2,\gamma)}{RDSS(g^2,\gamma)}$
Adam	$RDSS(g,\beta)$	$\frac{\eta}{RDSS(g^2,\gamma)}$

$$\Delta^{(k)} = -\eta^{(k)} G^{(k)}$$
$$\theta_{k+1} = \theta^{(k)} + \Delta^{(k)}$$

where $RSS(f)$ (Rooted Sum of Squares) e $RDSS(f,\gamma)$ (Rooted Damped Sum of Squares) are defined as

$$RSS(f) = \sqrt{f^{(0)} + f^{(1)} + \cdots + f^{(k-1)} + f^{(k)} + \varepsilon}$$
$$RDSS(f,\gamma) = \sqrt{(1-\gamma)(\gamma^k f^{(0)} + \gamma^{k-1} f^{(1)} + \cdots + \gamma f^{(k-1)} + f^{(k)}) + \varepsilon}$$

Second-Order Methods

The search for the minima of an objective function $J(\theta)$ is fundamentally a search for points where the gradient vanishes, i.e., $\nabla J(\theta) = 0$. This stationary condition allows for the adaptation of root-finding algorithms, most notably the **Newton-Raphson method**, directly to the gradient vector field.

Newton's method exploits both first- and second-order information by approximating the objective function locally with a quadratic model. In one dimension, the iteration reads

$$\theta_j^{(k+1)} = \theta_j^{(k)} - \left.\frac{J'(\theta)}{J''(\theta)}\right|_{\theta^{(k)}}. \tag{3.70}$$

In higher dimensions, the update takes the form

$$\theta^{(k+1)} = \theta^{(k)} - \left.\left(H(J)^{-1}\nabla J\right)\right|_{\theta^{(k)}}, \tag{3.71}$$

where

$$H(J)_{ij} = \frac{\partial^2 J}{\partial\theta_i \partial\theta_j} \tag{3.72}$$

is the **Hessian matrix** of second-order partial derivatives.

Geometrically, Newton's method refer at each step to a better approximation of $J(\theta$ around $\theta^{(k)}$ to move to the next point $\theta^{(k+1)}$.

- Gradient Descent locally approximates $J(\theta)$ with a plane tangent at $\theta^{(k)}$ and moves on such a plane in the direction of steepest descent (perpendicular to the level sets) by a step equal to the slope (or in general proportional to it by the learning rate η)
- Newton's method approximates $J(\theta)$ with parabolic surface tangent at at $\theta^{(k)}$ and takes the minimum of such surface as the next point $\theta^{(k+1)}$. This results in quadratic convergence near the optimum, significantly outperforming the linear convergence of first-order methods. Since this could lead to long steps ($\theta^{(k+1)}$ far away from $\theta^{(k)}$) which could be risky if the approximation of $J(\theta)$ by a quadratic surface is poor, the use of a damping factor $\eta < 1$ is useful here.

$$\Delta^{(k)} = -\left(H(J)^{-1}\nabla J\right)\Big|_{\theta^{(k)}}$$
$$\theta^{(k+1)} = \theta^{(k)} + \eta\Delta^{(k)}$$

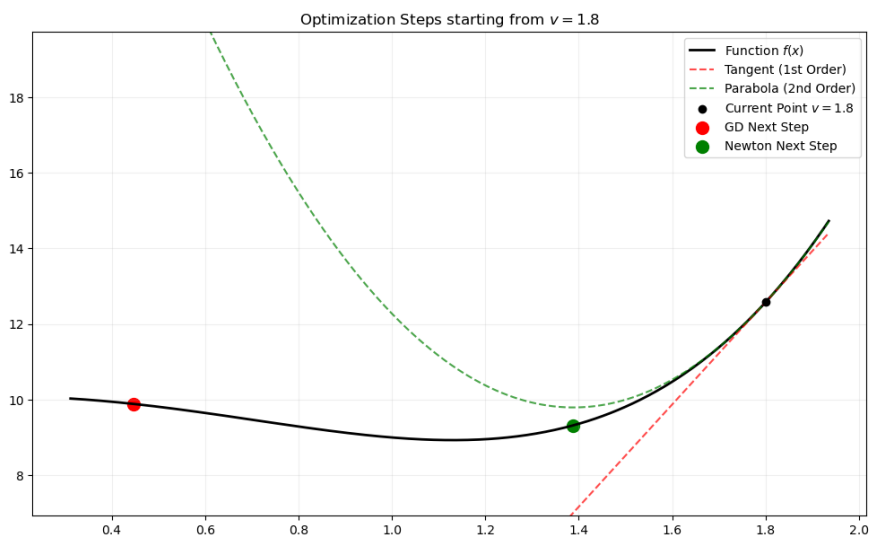


Figure 3.17. Comparison of GD and Newton's method function approximation

Unfortunately, the computational overhead introduced by this method is substantial:

- In terms of memory, since storing the Hessian requires $O(n^2)$ space, which is prohibitive for high-dimensional parameter spaces.
- In terms of computation, because inverting the Hessian (or solving the linear system $H\Delta = -\nabla J$) scales as $O(n^3)$.

To bridge the gap between efficiency and curvature awareness, Quasi-Newton methods (such as BFGS or L-BFGS) approximate the inverse Hessian $H(J)^{-1}$ with a symmetric, positive-definite matrix $B(J)$. This matrix is updated at each step using only gradient information.

These methods offer a superior balance, achieving superlinear convergence without the prohibitive cost of explicit second-order derivatives, making them practical for large-scale machine learning problems.

Optimization in deep learning

Second order methods like L-BFGS seem clearly preferrable to first order ones, such as Adam. By utilizing the Hessian matrix H, these methods account for the curvature of the loss landscape, allowing for a much more direct route to the minimum than the "blind" descent of first-order gradients. However, in the context of modern deep learning, first-order methods - specifically **Adam** and its variants - remain the undisputed workhorses.

The most immediate barrier to second-order methods is the sheer scale of modern neural networks. An update in a second-order scheme typically involves the inverse of the Hessian, or an approximation thereof. If a model has n parameters, the Hessian contains n^2 entries. For a transformer with hundreds of billions parameters, storing or inverting such a matrix is computationally impossible.

Even L-BFGS, which cleverly avoids storing the full matrix by using a history of gradient updates to approximate the curvature, requires multiple passes and line searches that are difficult to parallelize across GPUs. In contrast, Adam maintains only the two decaying sums $\tilde{G}, \tilde{H}$ for each parameter. This keeps the memory overhead linear $O(n)$, which is the maximum luxury we can afford when our models already push the limits of VRAM.

As a second point, remind that deep learning thrives on Mini-batch Gradient Descent. We rarely compute the true gradient of the entire dataset; instead, we estimate it using mini-batches. As observed, this introduces "noise" into the optimization process. Second-order methods are notoriously sensitive to this noise. The Hessian describes the local curvature, and if the local estimate is derived from a small, noisy batch, the "correction" suggested by a second-order method can be wildly inaccurate. Adam, however, is designed to be robust. By using the decaying sums $\tilde{H}$ and $\tilde{G}$, it essentially performs a low-pass filter on the noisy gradients, smoothing out the path toward the optimum in a way that second-order methods struggle to do without exact information.

Third, while a common misconception is that local minima are the primary enemy in deep learning, in high-dimensional spaces, the real villains are saddle points. Second-order methods are notoriously attracted to these points because they are designed to be "zero-gradient hunters."

To see why, consider the Newton update $\Delta\theta = -H^{-1}g$. We can decompose the Hessian using its eigenvectors q_i and eigenvalues λ_i:

$$H = \sum_{i=1}^{n} \lambda_i q_i q_i^T \tag{3.73}$$

The update in the direction of the i-th eigenvector is given by:

$$\Delta\theta_i = -\frac{1}{\lambda_i} g_i \tag{3.74}$$

If the curvature in the of direction q_i is positive this implies that $\lambda_i > 0$ and the step moves against the gradient, toward the point of 0 gradient, a minimum, with respect to the direction q_i. However, this happens also if the curvature is negative $\lambda_i < 0$, since in this case, instead of moving away from the peak, the Newton step moves *with*

the gradient, effectively climbing the hill to reach the exact point where the gradient vanishes. So, around a local minimum or maximum the method iterates up to the minim or maximum, the zero gradient point in both cases. The same happens in the case of saddle points, where the curvature is positive in some directions and negative in others.

Consequently, while Gradient Descent naturally moves away from maxima and saddle points by following the downward slope in some directions, a pure Newton method targets the point with null gradient, which might be a maximum of center of the saddle. It takes the local quadratic approximation too literally, treating the negative curvature as a peak to be summited rather than a slope to be descended.